



Ain Shams University
Faculty of Computer & Information Sciences
Information Systems Department

Smart Skill-Based Team Formation Platform for Hackathons (Team Catalyst)

**This documentation is submitted as required for the degree of bachelor's in
computer and information sciences.**

By:

Esraa Ahmed Taha [IS]

Basmala Ayman Zaky [IS]

Habiba Adel Abdelgowad [IS]

Hafsa Tarek Mohamed [IS]

Ruba Abdelsalam Abdelhameed [IS]

Zeina Mohamed Wady [IS]

Under Supervision of:

Dr. Naglaa Fathy

Lecturer,

Information Systems Department,

Faculty of Computer and Information Sciences,

Ain Shams University.

TA. Verena Nashat

Assistant Lecturer,

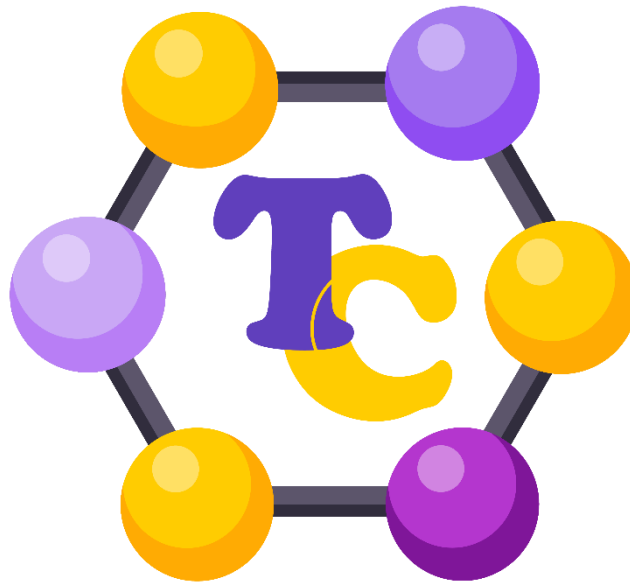
Information Systems Department

July 2026



Ain Shams University
Faculty of Computer & Information Sciences
Information Systems Department

Smart Skill-Based Team Formation Platform for Hackathons (Team Catalyst)



July 2026

Acknowledgement

We extend our heartfelt appreciation to Dr. Naglaa Fathy and TA. Verena Nashat for their unwavering support and guidance during our project. Their profound expertise and valuable insights have contributed to our project's accomplishment, and their dedication and commitment to our academic journey. Their mentorship and encouragement have played an instrumental role in our personal and professional growth. Thank you, Dr. Naglaa Fathy and Dr. Verena Nashat, for your invaluable assistance and belief in our potential.

Abstract

Team Catalyst is a smart web-based platform designed to improve the process of forming effective teams for hackathons and collaborative competitions. The system focuses on building well-balanced teams based on participants' skills, and experiences, while also considering their compatibility and potential collaboration dynamics.

By analyzing user profiles and available data, the platform generates team compositions that aim to achieve both diversity and cohesion helping participants form teams that are not only technically strong but also well-coordinated.

In addition, Team Catalyst provides an interactive environment where users can explore potential teammates, understand team structures, and adjust team compositions when needed. The platform supports flexible team formation, allowing participants to refine their teams according to changing goals and competition requirements.

By combining intelligent analysis with a user-centered approach, **Team Catalyst** enhances team formation in hackathons, enabling participants to collaborate more effectively and maximize their chances of success. The proposed system achieved promising results, with **68.84% Mean Precision**, **95.72% Mean Recall**, and **60.38% Mean Inter-List Diversity**, demonstrating its capability to form accurate, compatible, and diverse teams for hackathon environments.

الملخص

Team Catalyst هي منصة ويب ذكية تهدف إلى تطوير وتحسين عملية تكوين فرق العمل في الهاكاثونات والمسابقات التعاونية. يركز النظام على تشكيل فرق متكاملة من خلال تحليل مهارات المشاركين وخبراتهم وخصائصهم الفردية، مع مراعاة درجة التوافق بينهم وإمكانية تحقيق تعاون فعال داخل الفريق.

تعتمد المنصة على تحليل بيانات المستخدمين لتوليد مقترحات لتشكيل فرق عمل تحقق توازنًا بين التنوع في المهارات والانسجام بين الأعضاء، بما يساهم في بناء فرق قادرة على تحقيق أداء متميز من الناحيتين التقنية والتنظيمية.

كما توفر المنصة بيئة تفاعلية تُمكن المستخدمين من استكشاف أعضاء الفرق المحتملين، وفهم هيكل الفريق، وإجراء التعديلات اللازمة على تشكيل الفرق بما يتناسب مع متطلبات المسابقة وتطور أهدافها. وتدعم المنصة مرونة عالية في عملية التشكيل، مما يسمح بإعادة ضبط الفرق وتحسينها بصورة مستمرة.

ومن خلال دمج أساليب التحليل الذكي مع تصميم يركز على احتياجات المستخدم، تساهم **Team Catalyst** في الارتقاء بآليات تكوين الفرق في الهاكاثونات، وتعزيز كفاءة التعاون بين المشاركين، وزيادة فرص تحقيق نتائج ناجحة.

Contents

Acknowledgement	i
Abstract	ii
المخلص	iii
List of Figures	vii
List of Tables	ix
List of Abbreviations	x
Chapter 1: Introduction.....	1
1.1 Problem Definition.....	2
1.2 Motivation	2
1.3 Objective.....	3
1.4 Methodology	3
1.5 Time Plan	4
1.6 Document Organization.....	5
:Chapter 2 Background.....	6
2.1 Related Work.....	7
2.2 Similar Applications.....	10
2.3 Graph Representation Learning.....	11
2.3.1 The Mechanics of Spatial Neighborhood Propagation.....	12
2.3.2 Structural Propagation Scales: One-Hop vs. Two-Hop Contexts	12
Chapter 3: System Architecture.....	14
3.1 System Overview	15
3.1.1 System Architecture	15
3.1.2 Functional Requirements	16
3.1.3 Non-Functional Requirements	18
3.1.4 System Users	19
3.2 System Analysis & Design	20
3.2.1 Use Case Diagram	20
3.2.2 Database Diagram	21

3.2.3 Team Formation Sequence Diagram.....	22
Chapter 4: System Implementation and Experimental Results	26
4.1 Dataset	27
4.1.1 Data Cleaning.....	28
4.2 Input Data Preprocessing - CsdRec Framework I	30
4.2.1 Graph Construction	30
4.2.2 Semantic Representation Learning.....	32
4.2.3 Bayesian Personalized Ranking Optimization.....	37
4.3 Team Formation Engine - CsdRec Framework II	39
4.3.1 Relevance Awareness	40
4.3.2 Skill Diversity Awareness.....	41
4.3.3 Members Matching Algorithm.....	42
4.4 Dynamic Runtime Configuration	45
4.5 Technologies Used	45
4.6 Hardware & Software Configuration	47
4.7 Experiments and Results	47
Chapter 5: Run the Application.....	50
5.1 Entry Page (Landing Page).....	51
5.2 User Registration	52
5.3 User Login.....	53
5.4 Password Reset.....	53
5.5 Home Page	55
5.6 Profile Completion	56
5.7 Create Team Request	57
5.8 Project Ideas Exploration.....	60
5.9 Hackathons Exploration	61
5.10 Interested Hackathons	62
5.11 Team Recommendation.....	62
5.12 Team Progress Track.....	64

5.13 Team Invitations	64
5.14 Tracking Team Progress as an Invited Member	65
5.15 Viewing Participant Profiles.....	66
5.16 Team Rematching.....	67
5.17 User Dashboard	69
5.18 Admin Dashboard.....	69
Chapter 6: Conclusions and Future Work	71
6.1 Conclusion.....	72
6.2 Future Work	72
References	74

List of Figures

Figure 3.1: System Architecture	16
Figure 3.2 Use Case diagram.	20
Figure 3.3 Database diagram.	21
Figure 3.4 Team Recommendation Sequence Diagram	23
Figure 3.5 Project idea Posting & Join Request Sequence Diagram	25
Figure 4.1 :performance visualization.....	49
Figure 5.1: Entry Page (Landing Page)	51
Figure 5.2 : User Registration	52
Figure 5.3: Email Verification	53
Figure 5.4: User Login	53
Figure 5.5: Password Reset - 1	54
Figure 5.6: Password Reset - 2	54
Figure 5.7: Password Reset - 3	54
Figure 5.8: Home Page - 1	55
Figure 5.9: Home page - 2	56
Figure 5.10: Home Page - 3	56
Figure 5.11: Home page - 4	56
Figure 5.12: Profile Completion	57
Figure 5.13: Create Team Request - step 1.....	58
Figure 5.14: Create Team Request - step 2.....	58
Figure 5.15: Create Team Request – Invite Members	59
Figure 5.16: Create Team Request - step 3	59
Figure 5.17: Create Team Request - step 4.....	60
Figure 5.18: Create Team Request - step 5	60
Figure 5.19: Project Ideas Exploration	61
Figure 5.20: Hackathons Exploration	62
Figure 5.21: Interested Hackathons	62
Figure 5.22: Team Recommendation - Notification.....	63
Figure 5.23: Team Recommendation - All Recommended Teams' Options.....	63
Figure 5.24: Team Progress Track.....	64
Figure 5.25: Team Invitations - Notifications	65
Figure 5.26: Team Invitations.....	65
Figure 5.27: Tracking Team Progress as an Invited Member.....	66
Figure 5.28: Viewing Participant Profiles	67
Figure 5.29: Team Rematching - 1	67

Figure 5.30: Team Rematching - Notification.....	68
Figure 5.31: Team Rematching - 2	68
Figure 5.32: User Dashboard	69
Figure 5.33: Admin Dashboard - 1	70
Figure 5.34: Admin Dashboard - 2	70

List of Tables

Table 1.1: Time Plan.....	4
Table 2.1 : Comparison of AI-based Team Formation Methods.....	9
Table 2.2: Similar Applications Comparison	11
Table 3.1: Non-Functional Requirements.....	18
Table 3.2: System Users	19
Table 4.1: Sematic Representation Learning Module's Hyperparameters	39
Table 4.2: Model Hyperparameters	40
Table 4.3: Technologies Used in Team Formation Module	45
Table 4.4: Results.....	48

List of Abbreviations

- **Adam**: Adaptive Moment Estimation optimization algorithm
- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **BPR**: Bayesian Personalized Ranking
- **CP-DRL**: Capability and Position-aware Deep Reinforcement Learning
- **CsdRec**: Collaborative Software Development Recommendation
- **CTA**: Call-To-Action
- **Dev**: Development
- **DQN**: Deep Q-Network
- **GCO-DQN**: Graph Neural Networks with Deep Q-Networks (considering communication factors)
- **GNN**: Graph Neural Network
- **ID**: Identifier
- **JS**: JavaScript
- **KNN**: K-Nearest Neighbors
- **MADDPG**: Multi-Agent Deep Deterministic Policy Gradient
- **MILD**: Mean Inter-List Diversity
- **ML/AI**: Machine Learning / Artificial Intelligence
- **MP**: Mean Precision
- **MR**: Mean Recall
- **NLP**: Natural Language Processing
- **PyG**: PyTorch Geometric
- **RL-GP**: Reinforcement Learning-assisted Genetic Programming
- **SVM**: Support Vector Machine
- **UI/UX**: User Interface / User Experience
- **URL**: Uniform Resource Locator
- **XGBoost**: Extreme Gradient Boosting

Chapter 1: Introduction

1.1	Problem Definition.....	2
1.2	Motivation	2
1.3	Objective.....	3
1.4	Methodology	3
1.5	Time Plan	4
1.6	Document Organization.....	5

Chapter 1: Introduction

1.1 Problem Definition

In recent years, there has been a significant increase in interest in hackathons and competitive events, particularly in the fields of engineering and computer science. These events are not only popular among students but are also increasingly adopted by companies seeking innovative, out-of-the-box solutions to real-world problems.

Despite this growth, a major challenge continues to face participants, especially students, which are forming the right team. While team formation may initially appear to be a simple task, in practice, building a well-balanced and compatible team is a complex and time-consuming process. In some cases, it may take months before a competition to assemble a team, and even after this effort, there is no guarantee that the resulting team will perform effectively or meet expectations.

This issue largely stems from the informal and random nature of current team formation methods. Participants often rely on social media posts or limited personal networks to find teammates, without having clear indicators of compatibility, skill alignment, or collaboration potential. As a result, individuals may struggle to find team members with the required skill sets or shared interests. Forming an unsuitable team can lead to wasted time and effort, poor collaboration, and suboptimal project outcomes. In many cases, team members may face difficulties working together, leading to inefficiencies, conflicts, or even project failure.

1.2 Motivation

With the advancement of artificial intelligence and data-driven systems, there is a growing opportunity to improve the team formation process by analyzing participants' skills, experiences, and compatibility. Such approaches can enable the creation of balanced and cohesive teams that are more likely to collaborate effectively and achieve better outcomes.

This project is motivated by the need to provide participants with a structured, intelligent, and efficient way to form teams, ultimately enhancing their hackathon experience and increasing their chances of success.

1.3 Objective

This work aims to enhance the hackathon experience by providing an intelligent and flexible team formation system that is able to:

- Recommend multiple suitable team options for users and allow them to freely explore and choose their preferred team composition.
- Form balanced and high-performing teams by considering accuracy, diversity, skill complementarity, and previous collaboration relationships between members.
- Support personalized team formation by enabling users to select preferred teammates and intelligently complete or replace team members when necessary.
- Allow users to publish project ideas with their requirements, collect join requests, and apply AI-based matching to build compatible and balanced project teams.
- Support hackathon-wide team formation by gathering interested participants into shared candidate pools and automatically generating optimized teams.
- Provide flexibility by allowing users to accept or reject recommended teams according to their preferences.
- Enable continuous exploration of hackathons and project ideas, with search and filtering capabilities based on hackathon name, tags, and hackathon type (official or user-created).

1.4 Methodology

The project combines various techniques and technologies to achieve intelligent and efficient team formation:

Web Scraping: To collect real-world hackathon data from Devpost [1], including teams, projects, members, and interactions between them.

Data Cleaning and Filtering: To ensure data **reliability and trust meaningful team structure** by removing incomplete records, and eliminating noisy or irrelevant data.

Semantic Encoding and Tensor Preparation: To preprocess and standardize textual data such as project descriptions, skills, and member profiles.

Semantic Representation Learning: To generate and refine member and project representations by combining semantic features (such as text and skills) with

direct and indirect relationships within the collaboration network, while emphasizing the most important contextual information through attention mechanisms.

Bayesian Personalized Ranking (BPR): To optimize the recommendation model by ranking more relevant and compatible matches higher than less relevant ones.

Relevance and Diversity Scoring: To evaluate team quality by ensuring both strong skill alignment (relevance) and variation among team members (diversity).

Relevant Member Pool Construction: To reduce the search space by selecting the most relevant potential team members before team generation.

Greedy Beam Search Algorithm: To generate multiple team recommendations by maximizing a compatibility function that balances relevance and diversity.

React, Bootstrap, Node.js, Express.js, FastAPI: For frontend development, backend API management, and integration with AI modules respectively.

1.5 Time Plan

Table 1.1: Time Plan

TASK	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
Planning									
Research									
Dataset & Models Collection									
UI/UX Design									
Recommendation engine (AI Model + Algorithms)									
Front-End Dev									
Back-End Dev									
Testing & Improvements									
Final Version									

1.6 Document Organization

This document is structured into six main chapters as follows:

Chapter 2 –Background:

Presents the theoretical background related to artificial intelligence, and team formation and member selection. It includes a literature review of previous systems and research papers, as well as a comparison with similar existing applications.

Chapter 3 – System Architecture and Methods:

Describes the architecture of the proposed system, detailing the functional and non-functional requirements, the system components, user roles, and the overall system design methodology.

Chapter 4 – System Implementation and Experimental Results:

Explains how each system module was implemented, including the data collection and recommendation processes, The technologies and tools used in implemented modules are also discussed.

Chapter 5 – Running the Application:

Illustrates the process of using the system, including user registration and profile setup, how users explore hackathons, create or join teams, how team recommendations are generated, optimized, and displayed, and how team personalization features (such as inviting members or completing teams) are handled.

Chapter 6 - Conclusion and Future Work:

Summarizes the key outcomes of the project and outlines potential enhancements for future development.

Chapter 2: Background

2.1	Related Work	7
2.2	Similar Applications	10
2.3	Graph Representation Learning	11
2.3.1	The Mechanics of Spatial Neighborhood Propagation	12
2.3.2	Structural Propagation Scales: One-Hop vs. Two-Hop Contexts	12

Chapter 2: Background

In collaborative environments such as hackathons, forming effective teams represents a challenging problem that involves balancing multiple factors, including skills diversity, and compatibility between team members. Traditional team formation methods are often informal and rely on limited personal networks, which may lead to inefficient or unbalanced teams. This has created a need for more intelligent and structured approaches to support the team formation process. Recent advancements in AI have provided promising solutions to this challenge. Techniques such as **representation learning** enable the transformation of users and projects into numerical representations (*embeddings*) that capture their key characteristics. Additionally, **graph-based approaches** allow modeling relationships between members, making it possible to consider both direct and indirect connections within a collaboration network.

Moreover, **semantic encoding and tensor preparation** techniques help in understanding and processing textual data such as member skills and project descriptions, while **modern learning methods** support ranking and selecting the most relevant and compatible team members. Together, these approaches enable more accurate and personalized recommendations.

In this chapter, we present some of the recent related studies in section 2.1, which are summarized in Table 2.1. Also, we illustrate, in section 2.2, common existing applications and compare them to our proposed system in table 2.2. Finally, an overview of the graph representation learning process adopted in our project is explained in section 2.3.

2.1 Related Work

Authors in [2] proposed a reinforcement learning-assisted genetic programming approach for the team formation problem. The model combines 0–1 integer programming, genetic programming, Q-learning, and a KNN surrogate model to improve efficiency. Experiments were conducted on synthetically generated datasets ranging from 25 to 225 positions. The results showed that the proposed approach achieves faster learning time, better total efficiency, and improved scalability compared to traditional methods. However, the study relied solely on synthetic data, and the use of KNN reduced accuracy, while the inherent randomness of genetic programming led to variability in results [2].

The work in [3] introduced a deep reinforcement learning approach for team formation in large organizations. The proposed model (GCO-DQN) integrates

graph neural networks with Deep Q-Networks to capture skills, rank, and communication factors. It was evaluated on both real-world (20,659 employees) and synthetic datasets (100k employees), using metrics such as skill overlap and communication distance. The results demonstrated superior performance compared to baseline methods, with noticeable improvements in team quality and runtime efficiency. However, the study faced limitations due to the complexity of real organizational environments and the restricted accessibility of the real dataset [3].

The study in [4] explored a graph-enhanced reinforcement learning approach for group formation in collaborative problem-solving scenarios. The model utilizes graph theory and multi-agent deep reinforcement learning (MADDPG) to optimize team interactions, based on a dataset collected from an internship program involving 48 engineering students. The results indicated improvements in connectivity, participation balance, and communication efficiency. However, the model simplifies real-world teamwork dynamics and does not adequately account for individual skills and personal attributes [4].

In [5], the authors presented an accuracy and diversity-aware team recommendation system for collaborative software development. The model leverages text-enhanced embeddings, two-hop graph learning, and diversity-aware optimization to generate high-quality team recommendations. It was evaluated using Kaggle and GitLab datasets, showing significant improvements in Mean Precision, Mean Recall, and Mean Inter-List Diversity.

In [6], machine learning techniques were applied to predict relationships between participants and improve team formation using Steiner tree optimization. The study utilized models such as XGBoost, SVM, and linear regression on patent datasets from IBM and Samsung. The results showed high prediction accuracy, particularly with XGBoost achieving up to 92%. [6].

Overall, existing approaches focus on optimizing either accuracy, structural relationships, or scalability, but often lack a comprehensive consideration of both diversity and user-driven customization, which motivates the need for more flexible and intelligent team formation systems.

Table 2.1 : Comparison of AI-based Team Formation Methods

Paper Tittle	Year	Techniques	Dataset	Remarks / Results
A Reinforcement Learning for Team Formation Problem [2]	2025	0–1 Integer Programming (for Individual representation) Genetic Programming and Q-learning (for Team formation)	Generated datasets of diverse sizes	RL-GP is faster. It finds better solutions. It scales better for large problems.
Team Formation in large organizations: Deep reinforcement learning approach [3]	2024	GNN (understands the graph) DQN (makes decisions. n-step) Q-learning (improves learning accuracy)	Real dataset: 20,659 employees Synthetic dataset: 100k employees (To test performance in diff conditions)	GCO-DQN = 1.0 (best) CP-DRL $\approx$ 0.96–0.99 Greedy & Random much worse Runtime is better than CP-DRL
Graph Enhanced Reinforcement Learning for Effective Group Formation in Collaborative Problem Solving [4]	2024	Graph Theory (represents people as nodes and their interactions as edges) MADDPG (learns the best way to adjust these connections) Clustering (divides people into suitable groups)	"RescuShell" Data, 48 engineering students collaborated on a project.	The AI agent successfully learned, increasing its score from 86 to over 120

Paper Tittle	Year	Techniques	Dataset	Remarks / Results
Accuracy and Diversity-Aware Team Recommendation for Collaborative Software Development [5]	2023	CsdRec: Text-enhanced Embeddings (Use text semantics) Two-hop Graph Learning (Refine representations) Diversity Learning (Ensure diverse skills)	Kaggle (Public): 549 competitions, 6,769 teams for team formation modeling. GitLab (Private): 1,058 tasks, 138 developers for model validation.	CsdRec shows major improvements: +18.88% MP, +10.63% MR, +21.60% MILD and on Kaggle even higher: +21.51%, +14.38%, +21.41%.
Using machine learning to team formation problem a cross-company study [6]	2020	XGBoost Linear Regression SVM (For Team Formation)	USPTO PatentsView database from IBM and Samsung	92% (XGBoost)

2.2 Similar Applications

1. Hackathon Camp

It is an online platform [7] designed to support hackathon participants in forming teams and collaborating on projects. It provides a space where users can join hackathons, showcase their skills, and connect with other participants. The platform focuses on manual team formation and networking, allowing users to browse profiles and select teammates based on interests and skills. However, it does not rely on AI-driven team matching, and the process largely depends on user initiative rather than intelligent recommendations.

2. HackathonParty

It is an online platform [8] that helps users participate in hackathons, discover them, and collaborate with others. It offers features like project showcasing,

communication tools, and basic team organization. While it supports collaboration and community engagement, it lacks advanced AI capabilities for forming optimized teams. Team creation is typically based on user selection or simple matching criteria rather than intelligent analysis of skills and compatibility.

3. HackTribe

It is a community-driven platform [9] aimed at connecting developers, designers, and innovators for hackathons and collaborative projects. It allows users to build profiles, highlight their skills, and join teams. Although it provides a better **community-based matching experience**, it still lacks **deep AI-based team formation**, such as analyzing personality traits, skill balance, or project requirements automatically.

Table 2.2: Similar Applications Comparison

System / Feature	AI Team Matching	Projects' ideas posting	Hackathons' Exploration
Hackathon Camp	✗	✗	✗
HackathonParty	✗	✗	✓
HackTribe	✗	✗	✓
Proposed System	✓	✓	✓

2.3 Graph Representation Learning

Traditional deep learning frameworks excel at processing data structured in linear sequences or rigid tabular matrices. However, complex collaborative ecosystems (such as member-project configurations and professional team dynamics) are inherently non-Euclidean. Attempting to flatten these network structures into isolated feature rows strips away the critical topological relationships that define how distinct entities influence one another. Graph Representation Learning resolves this limitation by mapping discrete entities and their physical linkages directly into continuous, low-dimensional vector spaces.

The primary objective of this framework is to compute highly descriptive latent vector representations (embeddings) that preserve both the localized

attributes of individual nodes and the global geometric layout of the entire network topology.

2.3.1 The Mechanics of Spatial Neighborhood Propagation

Rather than analyzing entities in total isolation, a graph-based framework updates representations by systematically diffusing information across connecting structural edges. Every discrete propagation layer executes three foundational processing phases:

1. **State Compilation:** Each unique node evaluates its current internal content attributes to compile an outbound structural feature message.
2. **Neighborhood Aggregation:** Connecting structural edges serve as pathways through which adjacent nodes transmit their features to a target entity. The target node collects these incoming vectors using mathematical operations (such as attention-weighted summations) to distill clear neighborhood context.
3. **Representation Updating:** The framework fuses the target node's baseline, localized features with the newly aggregated structural neighborhood context, adjusting its coordinates within the embedding space to reflect its network environment.

2.3.2 Structural Propagation Scales: One-Hop vs. Two-Hop Contexts

The predictive power and topological awareness of a graph architecture scale based on the depth of its message-passing iterations, moving outward through distinct spatial tiers:

- **One-Hop (First-Order) Interaction:** This foundational layer operates exclusively across localized network boundaries. Information is shared only between entities that possess a direct, explicit historical connection. This tier combines discrete transactional identity tokens with immediate semantic attributes,

capturing localized features to construct baseline profiles. While highly accurate regarding explicit interactions, one-hop structures function in isolation from the broader network structure.

- **Two-Hop (Second-Order) Graph Networks:** To overcome data sparsity and cold-start bottlenecks, the aggregation scope is extended to second-order neighborhoods to capture high-order propagation dependencies. A two-hop aggregation routing enables a target node to distill information from its neighbors-of-neighbors. In a bipartite ecosystem, this tier uncovers latent, indirect relationships such as co-participation patterns (e.g., establishing a structural similarity between two developers because they have contributed to the exact same historical project tasks).

By transitioning from localized textual lookups into non-local topological neighborhoods, the framework filters out historical noise, mitigates sparse network constraints, and uncovers underlying semantic commonalities across entirely disconnected components of the topology. These multi-tier spatial vectors are ultimately combined via dense concatenation operators, yielding highly enriched vector matrices that feed downstream relevance scoring and optimization layers.

Chapter 3: System Architecture

3.1	System Overview.....	15
3.1.1	System Architecture	15
3.1.2	Functional Requirements	16
3.1.3	Non-Functional Requirements.....	18
3.1.4	System Users	19
3.2	System Analysis & Design.....	20
3.2.1	Use Case Diagram.....	20
3.2.2	Database Diagram.....	21
3.2.3	Team Formation Sequence Diagram	22

Chapter 3: System Architecture

3.1 System Overview

In this section, a detailed explanation of the system's architecture and a discussion of both functional and non-functional requirements are presented.

3.1.1 System Architecture

The proposed system aims to facilitate **intelligent team formation for hackathons** by leveraging AI techniques to analyze user skills, project requirements, and compatibility. The architecture consists of multiple interconnected layers, including the **presentation layer, application layer, and data layer**, *Figure 3.1* illustrates the high-level architecture of the system.

The process begins when users interact with the front-end interface through the presentation layer, where they can discover available hackathons, specify team requirements, and receive recommended teams. The user inputs, including skills, preferences, and project-related requirements, are then transferred to the application layer for processing and analysis.

Within the application layer, the system first performs input data preprocessing to prepare the collected data for intelligent analysis. During this stage, graph construction techniques are applied to model the relationships between users, skills, and hackathon requirements in the form of interconnected nodes and edges. Afterwards, semantic representation learning is used to generate meaningful representations of users and their skills, enabling the system to better understand similarities and compatibilities between participants. The processed data then undergoes BPR (Bayesian Personalized Ranking) optimization to improve the ranking quality of recommended matches based on user preferences and interaction patterns.

After preprocessing and optimization, the data is passed to the team formation engine, which represents the core component of the proposed system. The engine performs relevance awareness analysis to ensure that selected members match the team and project requirements. In addition, skill diversity awareness is applied to create balanced teams with complementary skills rather than teams with duplicated capabilities. Finally, the members matching algorithm analyzes all processed information to generate optimized team recommendations by selecting the most compatible participants for each team.

The system relies on a dedicated data layer that stores and manages all required

information through multiple databases, including the hackathons' database, teams' database, and members' database. These databases provide the necessary information for the matching and recommendation processes and continuously update the system with user and team data.

The **final output** of the system is a set of AI-based recommended teams that are returned to the users through the front-end interface. These recommendations are generated based on skill compatibility, relevance to project requirements, and team balance, allowing users to efficiently form competitive and well-structured hackathon teams.

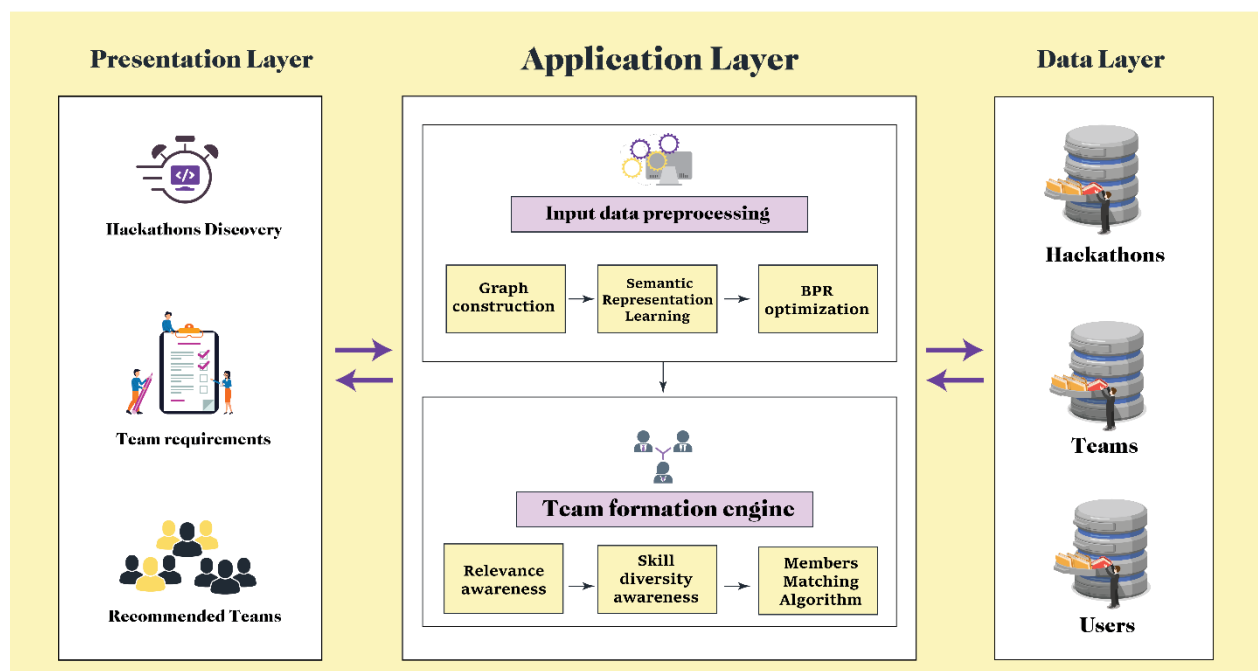


Figure 3.1: System Architecture

3.1.2 Functional Requirements

The system is expected to fulfill the following functional requirements:

1. User Authentication

- Enable users to register, log in, and securely manage their profiles and skills information.

2. Hackathon Discovery

- Allow users to browse and explore available hackathons.

3. Profile and Skills Management

- Enable users to add, edit, and manage their skills, experience level, and interests.

4. Team Creation and Requirements Input

- Allow users or team leaders to specify required roles, skills, team size, and preferences for hackathon participation.

5. Project Idea Posting

- Enable users to create teams with project ideas by publishing project descriptions, required technologies, and expected team requirements.

6. Explore Projects

- Allow users to browse and explore published project ideas that are open for participation.

7. Candidate Pool Generation

- Collect interested participants into candidate pools for hackathons or project-based teams before executing the matching process.

8. Partial Team Completion

- Enable users to create or join partially completed teams by inviting team members and allowing the system to intelligently recommend additional members needed to complete the team.

9. Hackathon Interest Registration

- Allow users to express interest in specific hackathons and participate in automated AI-based team formation processes.

10. AI-Based Members Matching

- Analyze users' skills, compatibility, interests, and project requirements to recommend suitable teammates automatically.

11. Skill Diversity and Compatibility Analysis

- Ensure balanced team formation by considering complementary skills while minimizing redundancy among team members.

12. Greedy-Based Team Generation

- Generate optimized hackathon teams using a Greedy Beam Search–based recommendation pipeline that balances member relevance, compatibility, and diversity while supporting personalized team formation and team completion scenarios.

13. Recommended Teams Display

- Present users with optimized team recommendations based on relevance, compatibility, and team balance scores.

14. Search and Filtering

- Search by hackathon name, filter by tags, hackathon state (official / user-created), filter recommended teams by (My created teams, suggested to join)

15. Notification Management

- Notify users about joining requests, accepted invitations, rejected requests, generated team recommendations, and team updates.

16. Full-Stack Integration

- Ensure smooth data exchange between the FastAPI, backend API (Node.js), and React frontend.

17. Database Management

- Store and manage user profiles, hackathon information, team data, and recommendation results efficiently.

3.1.3 Non-Functional Requirements

Table 3.1: Non-Functional Requirements

Requirement	Description
Performance	The system must generate team recommendations within a short response time after receiving user input.
Scalability	The platform should support a large number of users, hackathons, and team requests simultaneously.
Usability	The user interface must be intuitive, responsive, and easy to use for participants with different technical backgrounds.
Interoperability	The frontend, backend, database, and AI modules must communicate seamlessly with each other.
Maintainability	The system architecture and codebase should be modular and easy to maintain or extend in the future.

Requirement	Description
Availability	The platform should remain accessible and operational with minimal downtime during hackathon activities.

3.1.4 System Users

Table 3.2: System Users

User Role	Description
Participant (Competitor)	The primary user of the system can create and manage a profile, add skills and interests, explore hackathons and project ideas, create or join teams, request team completion, and receive AI-generated team recommendations.
Team Leader	A participant who can create teams, publish project ideas, define required skills and roles, manage team members, review join requests, and request intelligent team recommendations and completion.
Administrator (Optional)	Monitors system performance, review users, hackathons, teams, and databases, recent activities.

3.2 System Analysis & Design

This section presents the architecture of the proposed platform using a set of diagrams that illustrate the system workflow, data relationships, and interactions between its components. These diagrams provide a clear understanding of how the platform operates and supports intelligent hackathon team formation.

3.2.1 Use Case Diagram

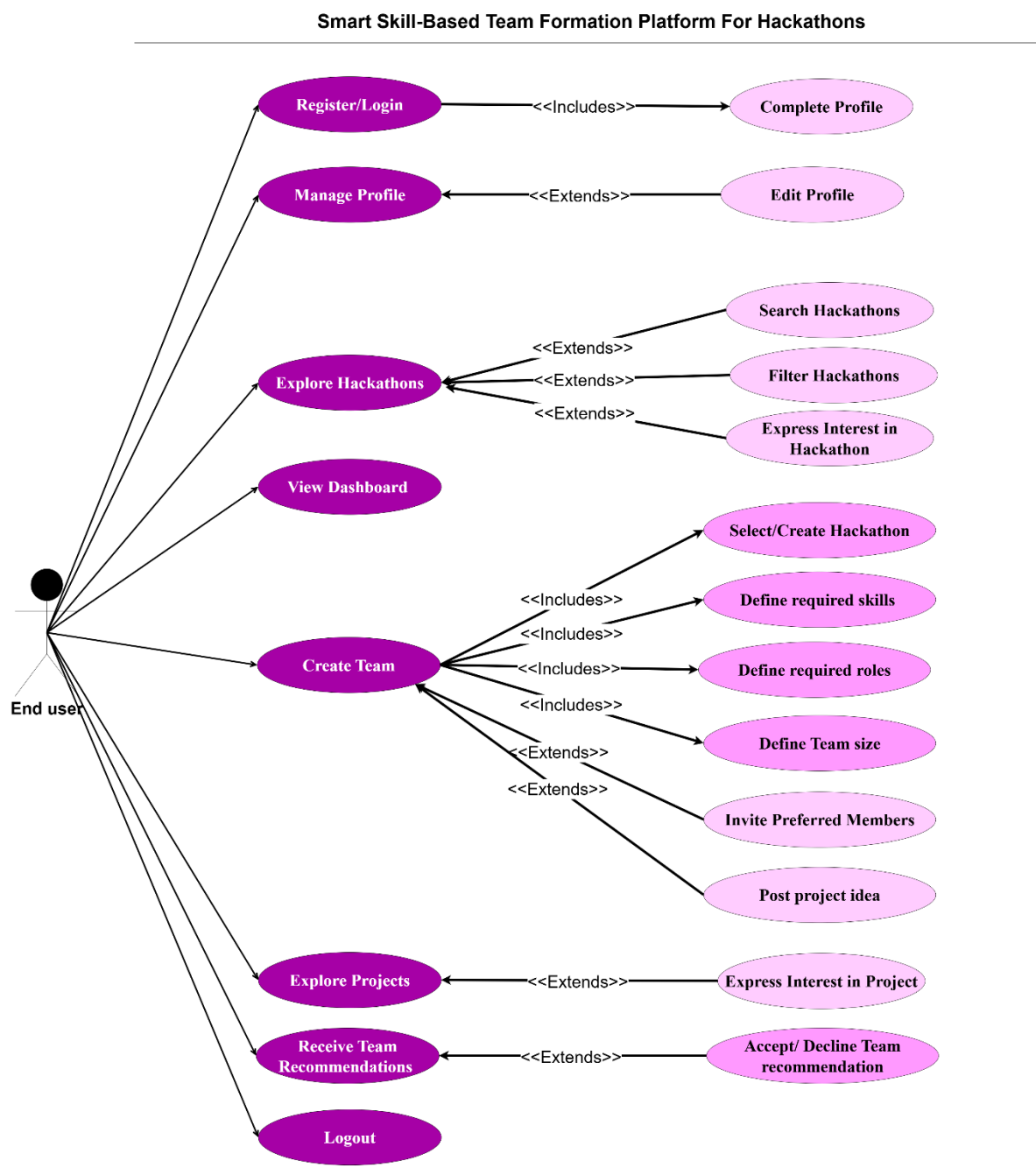


Figure 3.2 Use Case diagram.

This use case diagram illustrates the core functionalities of the Smart Skill-Based Team Formation Platform for Hackathons from the **end user's perspective**. The system enables users to register and authenticate securely, manage and complete their profiles, and explore available hackathons using search and filtering features. Users can express interest in hackathons and access a personalized dashboard to monitor their activities and team-related updates.

The platform also allows users to create teams by selecting the hackathon, defining required skills, and providing team size. Additionally, users can optionally invite preferred members or post project ideas to attract suitable participants. Users may request to join existing projects and receive AI-generated team recommendations based on skills compatibility, relevance, and team balance. Finally, users can accept or decline the recommended teams, providing an intelligent and interactive environment for forming effective hackathon teams.

3.2.2 Database Diagram

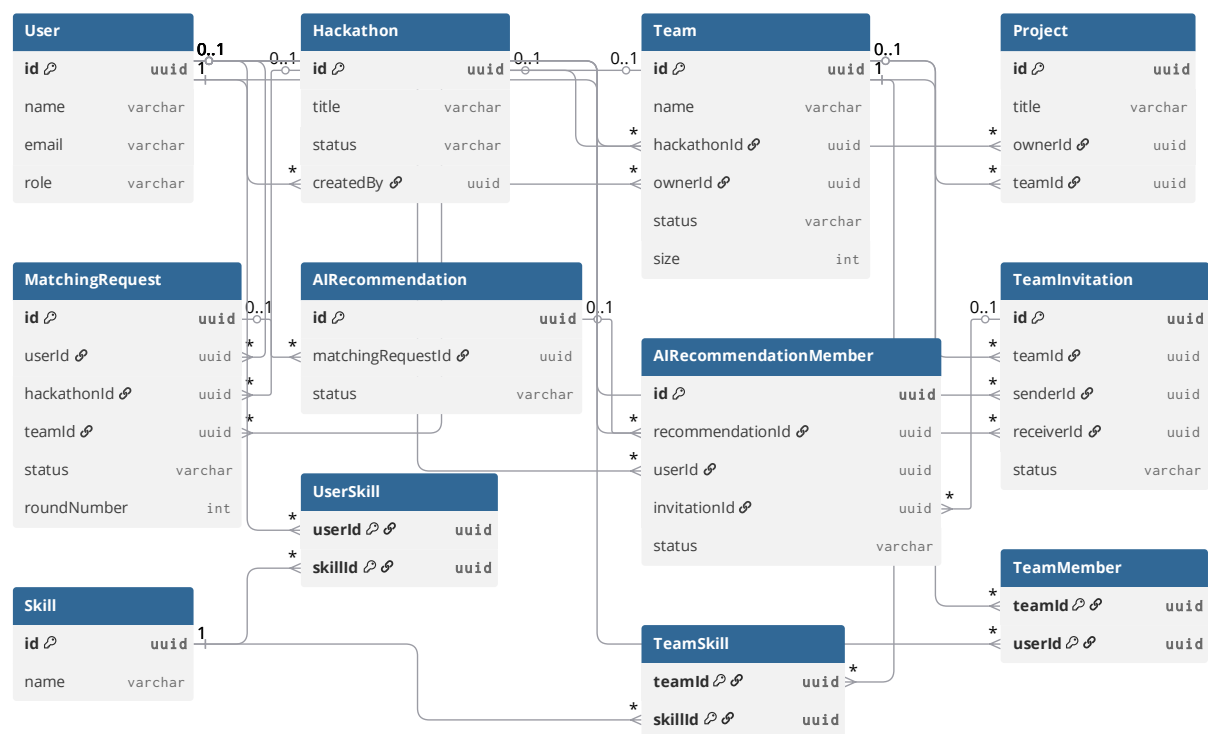


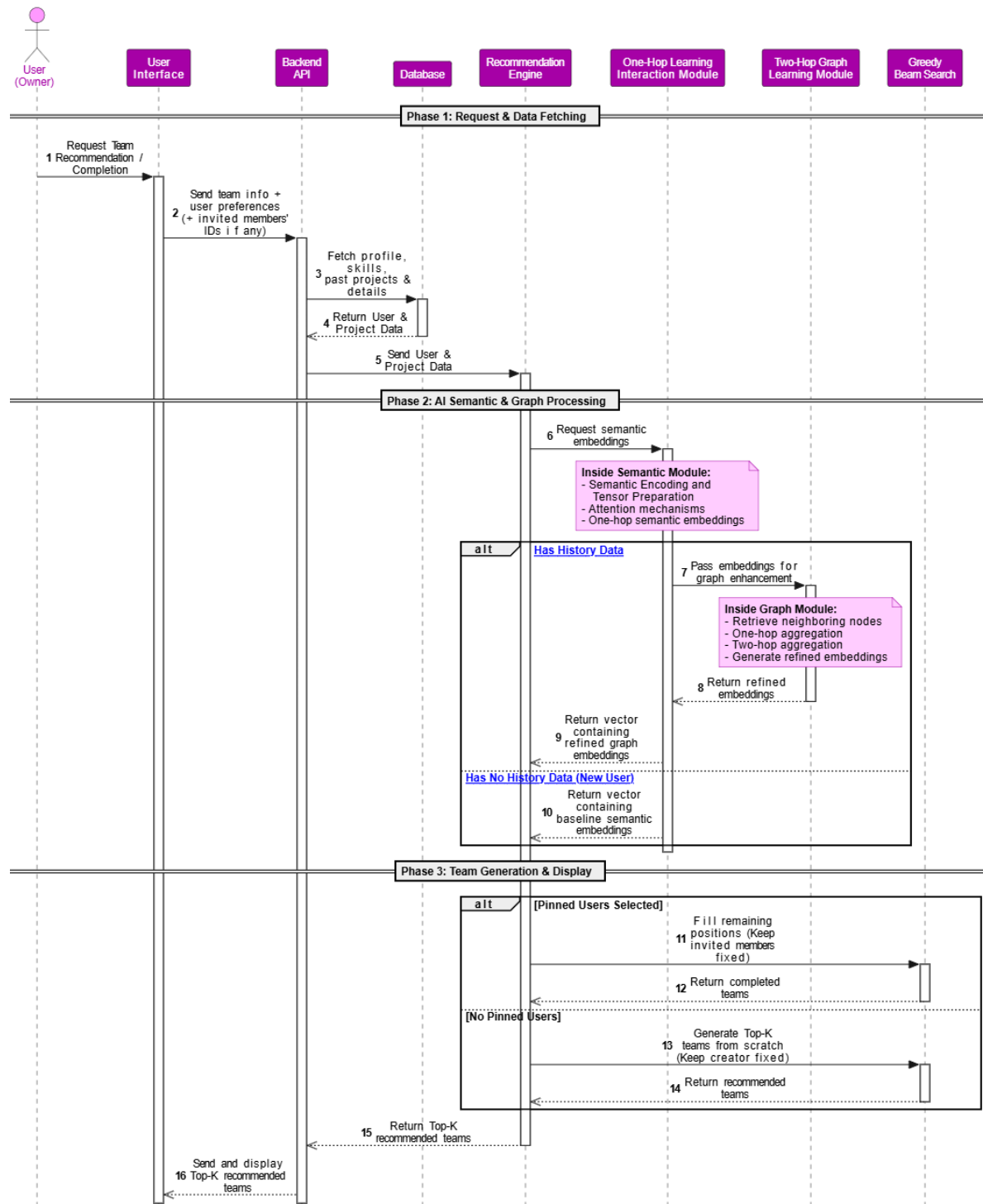
Figure 3.3 Database diagram.

This database diagram represents the overall architecture of the Hackathons Team Formation system. It includes modules for user authentication, profiles, skills,

hackathons, projects, teams, interests, notifications, invitations, and AI-powered matching. The relationships between entities are designed to support efficient collaboration, automated recommendation workflows, and seamless team formation for both hackathons and user-created projects.

3.2.3 Team Formation Sequence Diagram

• Team Recommendation Sequence Diagram



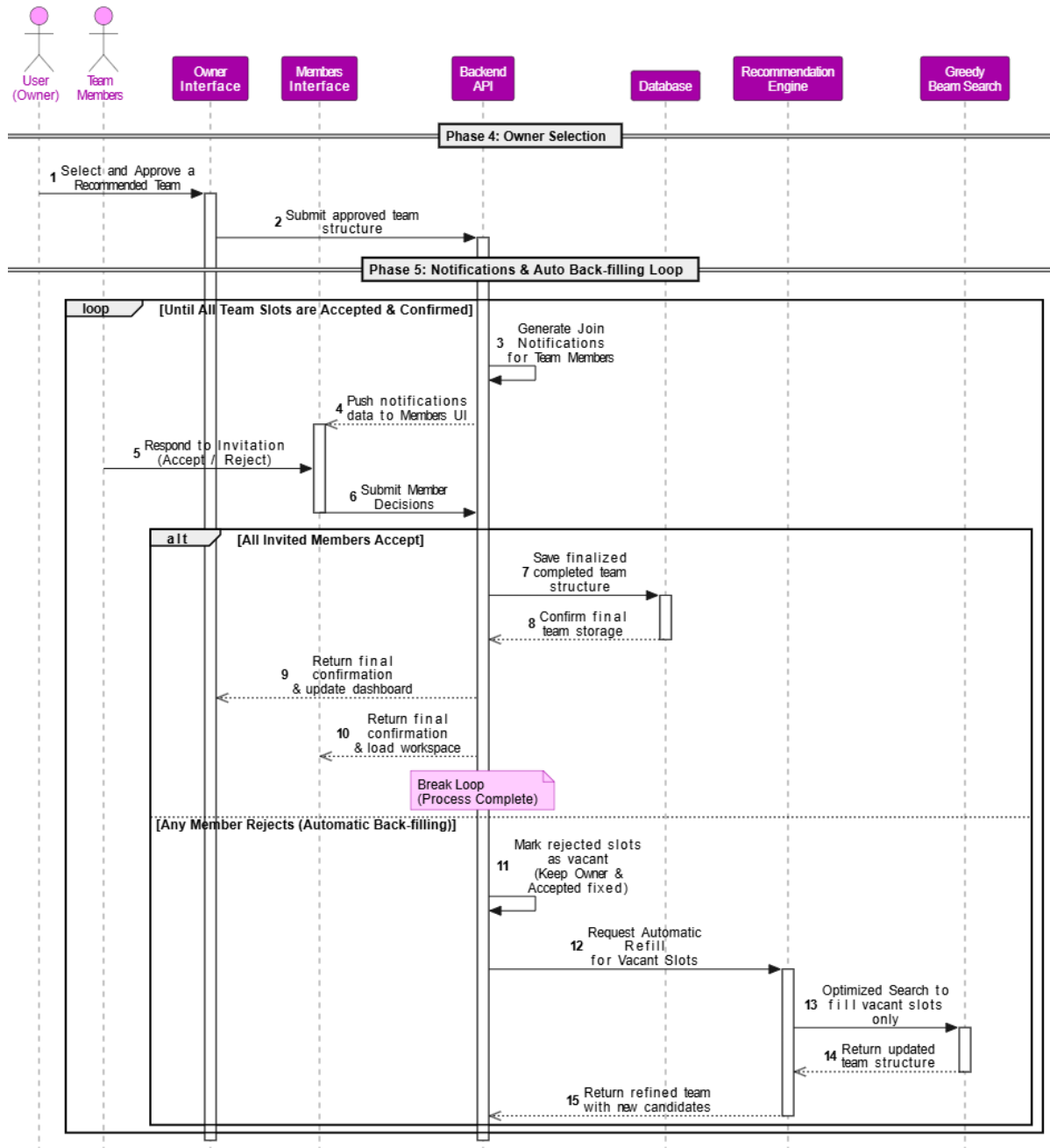
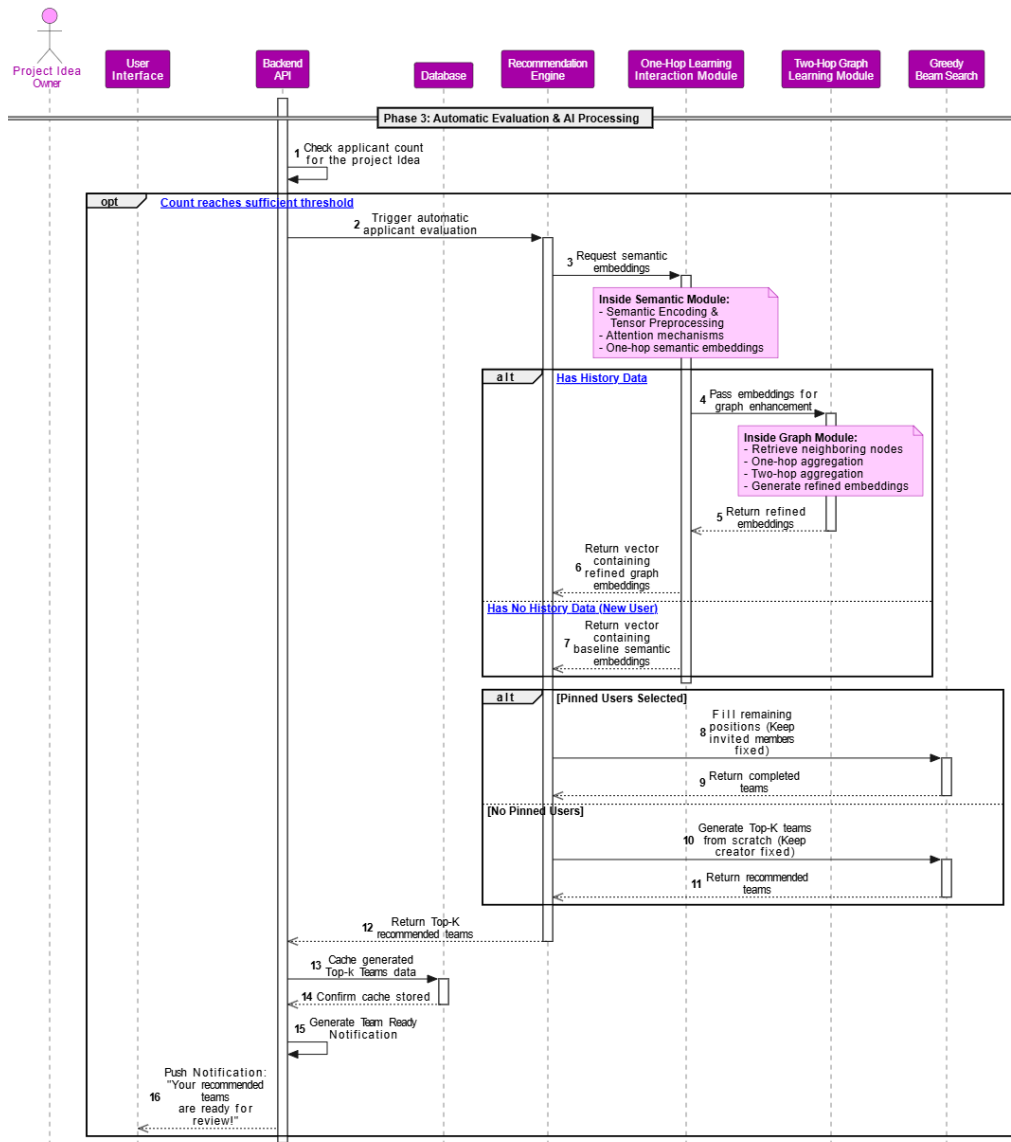
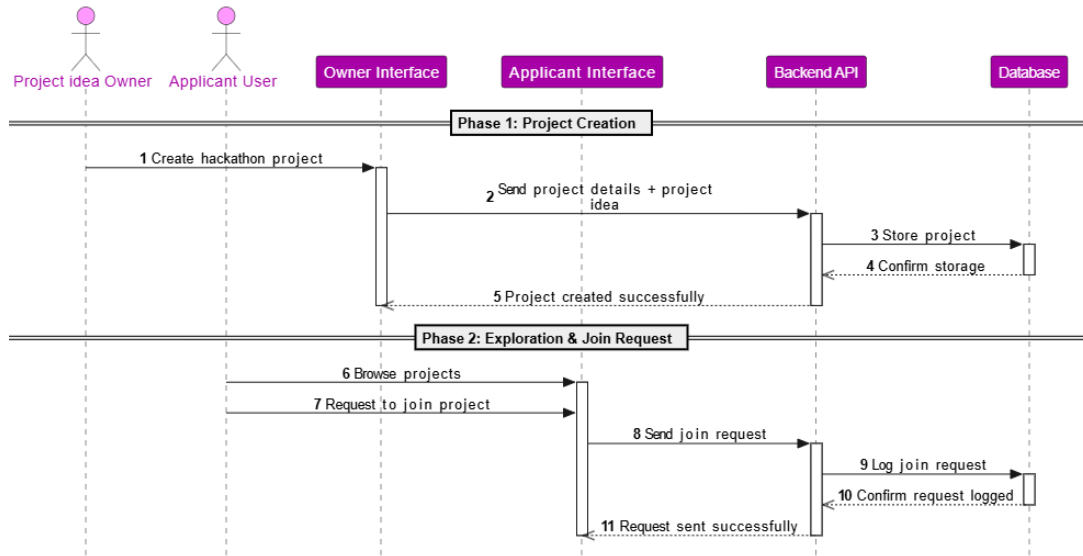


Figure 3.4 Team Recommendation Sequence Diagram

This sequence diagram demonstrates how the system generates a comprehensive team recommendation or completes an existing team based on user preferences. It begins with the user submitting team details and user preferences (along with invited members, if any), followed by data retrieval from the database. The system then processes this data through the One-Hop Interaction Learning and Two-Hop Graph Learning modules to generate refined embeddings. Finally, the Recommendation Engine utilizes Greedy Beam Search to evaluate candidates under two distinct alternatives, either filling vacant slots around pinned members or generating a top-K team from scratch before returning the finalized recommendation to the user interface.

• Project idea Posting & Join Request Sequence Diagram



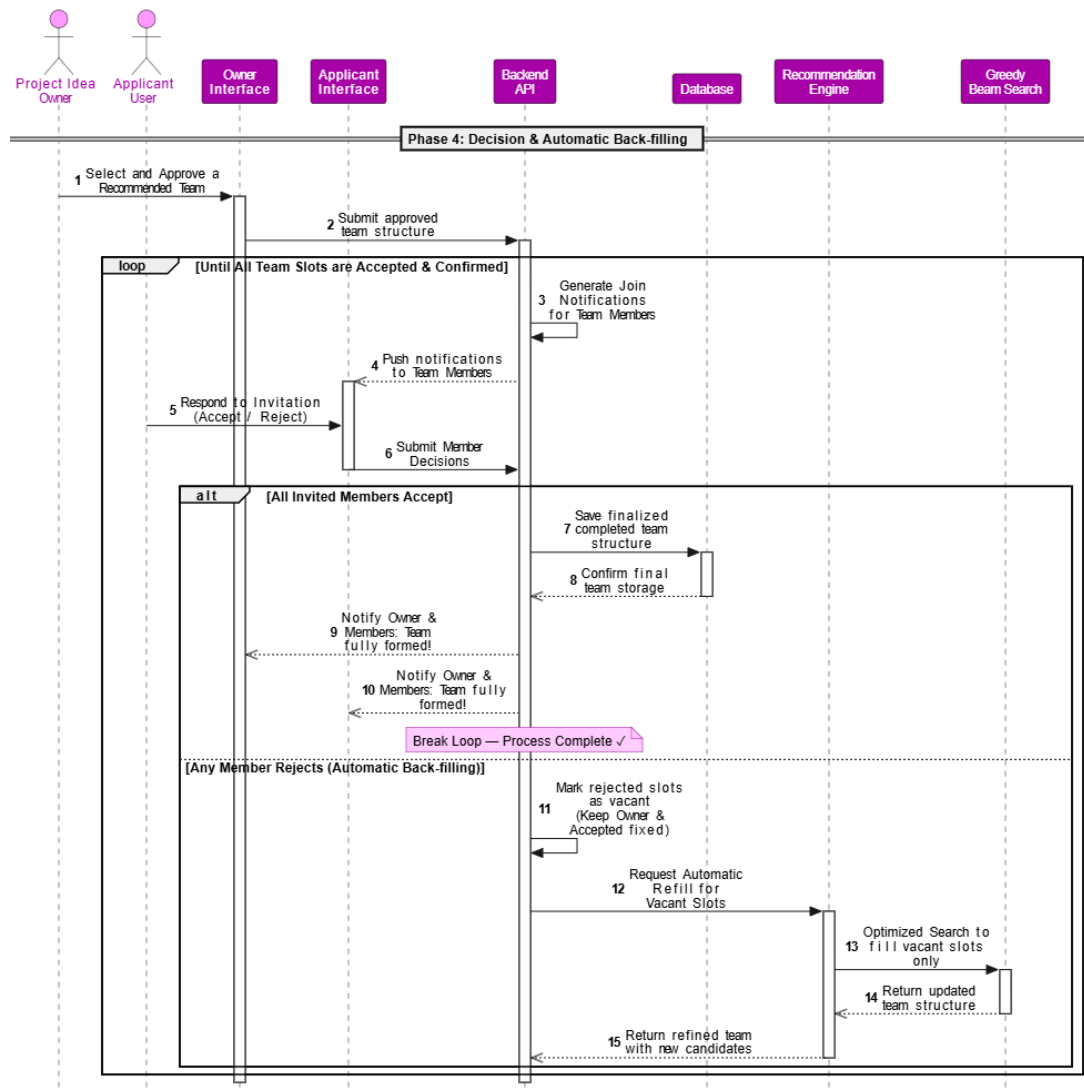


Figure 3.5 Project idea Posting & Join Request Sequence Diagram

This sequence diagram illustrates the lifecycle of project creation, and the direct evaluation of applicant join requests, **specifically in the case where the Project Owner has a predefined project idea**. It begins with the Project Owner posting this specific project idea, which is successfully logged into the database, followed by an Applicant User browsing and submitting a join request to this particular project. When the Owner accesses the project dashboard, the Recommendation Engine evaluates and ranks the applicants based on embedding compatibility. Finally, the diagram demonstrates the centralized decision phase, where the Project Owner exercises direct approval over the recommended team, triggers immediate database storage, and dispatches official assignment notifications to the accepted team members.

Chapter 4: System Implementation and Experimental Results

4.1	Dataset	27
4.1.1	Data Cleaning.....	28
4.2	Input Data Preprocessing - CsdRec Framework I	30
4.2.1	Graph Construction	30
4.2.2	Semantic Representation Learning.....	32
4.2.3	Bayesian Personalized Ranking Optimization.....	37
4.3	Team Formation Engine - CsdRec Framework II	39
4.3.1	Relevance Awareness	40
4.3.2	Skill Diversity Awareness.....	41
4.3.3	Members Matching Algorithm.....	42
4.4	Dynamic Runtime Configuration	45
4.5	Technologies Used	45
4.6	Hardware & Software Configuration	47

Chapter 4: System Implementation and Experimental Results

In this chapter, we start by explaining the initial dataset collection, multi-relational graph construction, and semantic preprocessing pipelines in the Dataset and Input Data Preprocessing sections. Then, the Team Formation Engine and concluding sections describe the relevance and diversity-aware recommendation algorithms, team matching algorithm, alongside the dynamic runtime and hardware configurations utilized to implement the final system.

4.1 Dataset

The dataset was constructed using a hybrid data collection strategy, where high-level hackathon directory data was fetched directly from the official Devpost API [10] while all remaining granular interaction records (including projects, team configurations, and individual profiles) were extracted via custom web scraping from Devpost [1]. By combining these two channels, the platform successfully captured a rich, multi-relational graph of interactions among hackathons, projects, teams, and individual members.

The dataset was constructed via web scraping from Devpost, capturing a rich, multi-relational graph of interactions among hackathons, projects, teams, and individual members. The dataset was constructed via web scraping from Devpost, capturing a rich, multi-relational graph of interactions among hackathons, projects, teams, and individual members [1].

Initially, the raw dataset was massive and highly noisy, encompassing 11,418 hackathons and over 345,000 projects. Because collaborative platforms naturally contain incomplete or ongoing data, a rigorous data cleaning protocol was applied to establish a reliable "ground truth". This included filtering for completed hackathons with announced winners, verified organizations, and removing private users or projects lacking sufficient descriptions. This distillation process yielded a high-quality, trustworthy subset consisting of 38,462 unique members across 18,569 projects.

Finally, to bridge the gap between raw platform data and the neural network's embedding layers, extensive Natural Language Processing (NLP) and tensor formatting were executed. Raw textual descriptions were tokenized, technology tags and skills were normalized, and comprehensive vocabulary mappings (word2id and label2id) were generated. The ultimate output of this pipeline is a set of model-ready tensors that successfully encapsulate the semantic text and

categorical labels necessary to power the framework's relevance and diversity algorithms.

The collected data was organized into five heavily detailed relational entities:

1. **Hackathons:** High-level directory data was captured via the web API [10], including Hackathon ID, Submission Period, Themes (tags), Prize Amount, Registrations Count, Sponsoring Organization, and whether the Winners were Announced.
2. **Hackathon Projects:** High-level project statistics were collected, containing the Hackathon ID, Project ID (Generated), Short Description, Participants Count, Likes Count, and Comments Count.
3. **Team & Project Details:** Detailed project and team attributes were scraped, including Hackathon ID, Project ID, Is Winner (True/False) status, Winners Description, Project Tags (Technologies), Short Description, and the full Project Description.
4. **Team Memberships:** The mapping between projects and users was recorded using the Project ID, Member ID (Generated), and the Member Description specific to their role in that project.
5. **Members (Users):** Comprehensive user profiles were built by extracting the Member ID, Skills, Hackathon Interests, specific Member Descriptions in projects, Projects Count, Hackathons Count, Achievements Count, Followers Count, Following Count, Likes Count, and total Winnings.

4.1.1 Data Cleaning

The raw collected dataset was massive, initially containing 11,418 hackathons and 345,804 projects. Because user-generated platforms inherently contain incomplete, spam, or ongoing entries, a rigorous multi-step filtration pipeline was executed to establish a reliable ground truth for the recommendation model.

The cleaning and filtering process applied in this project follows the same methodology proposed in the study by Nurhayati et al. [11], which performed preprocessing and cleaning on the same Devpost hackathon dataset:

1. **Status Filtering:** The dataset was strictly limited to hackathons that had completely "ended" (reducing the count to 11,262 hackathons).
2. **Hackathon Integrity:** Hackathons lacking a known organization (to maintain trust) or lacking specific themes were entirely dropped from the dataset. Additionally, small-scale hackathons with fewer than five submitted projects were removed and further filtered to only include those

that had explicitly announced winners to ensure trust and data integrity (resulting in 7,172 hackathons and 285,678 projects).

3. **Team Constraint Optimization:** Any team containing at least one private user was dropped to ensure full visibility of the collaboration graph.
4. **Project & Member Quality Control:** Projects missing essential semantic data, such as project tags or short descriptions, were eliminated, and any member lacking listed skills was dropped.

- **Final Dataset Statistics:**

After executing the rigorous cleaning pipeline, the final high-quality dataset size was finalized as follows:

- Total Hackathons: 2,742
- Total Projects/Teams: 18,569
- Total Team Memberships (Interactions): 66,553
- Total Unique Members: 38,462

- **Sample of the Cleaned Dataset:**

- **Cleaned Hackathons**

	hackathon_id	themes	prize_amount	registrations_count	projects_count
6	14985	['Beginner Friendly', 'COVID-19', 'Social Good']	216	23	6
8	13144	['Beginner Friendly', 'Machine Learning/AI', '...]	550	23	7
9	21461	['Education', 'Productivity', 'Social Good']	0	22	6

- **Cleaned Hackathon Projects**

	project_id	hackathon_id	short_description	likes	comments
0	0	20191	By taking advantage of Fitbit's technology, ou...	0	0
1	1	17776	Next Gen Web3 and AI powered All-in-one works...	5	1
2	2	18173	All-in-one workspace for Web3 companies and DAOs	0	0

- **Cleaned Team & Project Details**

	project_id	hackathon_id	members_count	is_winner	winners_description	project_tags	project_description	short_description
5119	0	20191	4	False	NaN	['css', 'firebase', 'flask', 'html', 'javascri...	Inspiration Our project began as we sat down t...	By taking advantage of Fitbit's technology, ou...
3270	1	17776	5	False	NaN	['cadence', 'chakra', 'css', 'next-auth', 'nex...	Link : Walkthrough of our cadence smart contra...	Next Gen Web3 and AI powered All-in-one works...
10137	2	18173	5	True	AWS Honorable Mentions\n ...	['13', 'next-auth', 'nextjs', 'postgresql', 'p...	Vision Get Paid for your Git Contributions- AI...	All-in-one workspace for Web3 companies and DAOs

- **Cleaned Team Memberships**

	project_id	member_id	member_description
0	0	15539	I worked on the Backend with flask and linking...
1	0	4474	NaN
2	0	7650	I was responsible for my teams artistic ventur...

- **Cleaned Members (Users)**

	member_id	bio	skills	hackathons_interests	Descriptions
0	0	NaN	[python, machine-learning, flask, django, fast...	['Machine Learning/AI', 'Productivity', 'Begin...	['I helped out with a mostly everything. I als...
1	1	High school student with a passion for applyin...	[python, javascript, java]	['Beginner Friendly', 'Low/No Code', 'Machine ...	NaN
2	2	NaN	[python, javascript]	['Cybersecurity', 'DevOps', 'Gaming', 'IoT', '...	['I used SAP BTP to implement and deploy the b...

4.2 Input Data Preprocessing - CsdRec Framework I

The following recommendation pipeline is inspired by the framework proposed in the CsdRec paper, “*Accuracy and Diversity-Aware Team Recommendation for Collaborative Software Development*” [5]. Several components of the original framework were adopted and extended to align with the objectives and requirements of the proposed hackathon team formation system.

4.2.1 Graph Construction

Following the methodology proposed in [5], to formally model the historical collaboration patterns between participants and historical projects, the system architecture constructs a structured **Bipartite Interaction Graph**. This topological network representation captures the multi-relational dependencies between members and the projects they previously contributed to, allowing the model to extract hidden collaboration dynamics and interaction properties.

- **Network Topology**

Let $G = (\mathcal{U}, \mathcal{V}, \mathcal{E})$ represent the **undirected, unweighted** bipartite collaboration network, where:

- $\mathcal{U} = \{u_1, u_2, \dots, u_m\}$ denotes the discrete set of unique user/developer nodes within the system database.
- $\mathcal{V} = \{v_1, v_2, \dots, v_m\}$ represents the discrete set of unique hackathon project or task nodes.

- $\mathcal{E} \subseteq \mathcal{U} \times \mathcal{V}$ defines the set of interaction edges mapping historical team memberships.

An edge $e_{ij} = (u_i, v_j) \in \mathcal{E}$ is instantiated if and only if member u_i has a verified historical interaction or registered contribution record with project v_j .

• Structural Adjacency and Graph Constraints

Because G is strictly governed by a bipartite schema, direct intra-set connections are structurally prohibited:

$$\mathcal{E} \cap (\mathcal{U} \times \mathcal{U}) = \emptyset \text{ and } \mathcal{E} \cap (\mathcal{V} \times \mathcal{V}) = \emptyset$$

Consequently, a developer cannot directly link to another developer, and a project cannot directly connect to another project within this isolated interaction layer. The structural topology of the network is mathematically preserved via a sparse adjacency matrix $\mathbf{R} \in \mathbb{R}^{m \times n}$, where individual matrix entries are deterministically defined as:

$$R_{ij} = \begin{cases} 1, & \text{if } (u_i, v_j) \in \mathcal{E} \\ 0, & \text{otherwise} \end{cases}$$

This discrete matrix formulation effectively packages the sparse historical edge distributions parsed from the raw web-scraped dataset.

• Purpose of the Graph Representation

The primary objective of mapping the collaboration space as a bipartite structure is to transform tabular team interactions into a structured relational graph. By representing the data through this network configuration, the system becomes capable of uncovering both direct and indirect collaboration relationships between platform members and projects.

Furthermore, this graph design serves as the mandatory structural foundation for implementing localized deep learning mechanisms:

- **One-Hop Learning Integration:** This tier directly captures immediate, explicitly documented interactions between members and their assigned

projects.

- **Two-Hop Learning Integration:** This tier expands the model's vision, allowing the system to execute message-passing operations that aggregate context from surrounding neighborhood nodes and indirect historical connections within the network.

Ultimately, converting raw database entries into a geometric graph framework allows the system to generate significantly richer member and project representation vectors. This deep contextual awareness directly translates into highly accurate team recommendations and robust relevance analytics.

4.2.2 Semantic Representation Learning

This module describes how the framework generates descriptive latent vectors from multi-stream textual and structural profiles, transitioning from raw data to localized content features, and ultimately to non-local topological neighborhoods. To bridge the gap between unstructured collaborative inputs and the structural graph layers, the system architecture operates as a multi-stage pipeline: it first engineers' uniform, model-ready input tensors through a deterministic preprocessing engine, and subsequently passes them to multi-tier neural layers for contextual and relational feature extraction.

• Semantic Encoding and Tensor Preparation

To interface the unstructured text fields and discrete categorical tags extracted from the collaboration platform with the deep learning layers of the neural recommendation network, the system executes a deterministic text normalization and vectorization pipeline. This preprocessing pipeline maps raw natural language characters into fixed-size numerical tensors capable of preserving semantic context and syntax boundaries during neural representation learning.

1. Text Tokenization

The system executes a systematic multi-stage normalization to clear linguistic noise from unstructured inputs (e.g., project descriptions and user biographies) before converting them into numerical vectors. The framework processes these data streams through the following concurrent operations:

- **String Standardization:** All raw text strings are lowercased, and uniform resource locators (URLs), hyper-links, and non-alphanumeric punctuation marks are stripped from the character stream.

- **Sequence Segmentation:** The filtered text character stream is segmented into discrete word tokens based on standard white-space boundaries.
- **Stop word Elimination:** The segmented token stream is filtered against a deterministic stop word collection to eliminate uninformative vocabulary syntax (e.g., articles, conjunctions).

2. Label Normalization

Unlike free-form textual descriptions, project tags and member skills required a distinct normalization mapping function (**clean_label**) to safeguard domain-specific programming syntax. Standard alpha-numeric filtering would inadvertently corrupt essential technical markers.

To resolve this, the normalization engine lowercases and trims whitespace strings while explicitly protecting critical programming language trailing symbols, ensuring identifiers such as "C++", "C#", and ".NET" are preserved with complete semantic integrity before vocabulary allocation.

3. Vocabulary Construction and Index Mapping

To project these discrete string tokens into structural lookup tables, the system architecture constructs two distinct, globally accessible dictionaries that map string keys to non-negative scalar integers:

- **word2id:** This mapping function processes the global set of unique natural language words discovered across user biographies and project descriptions, translating each token into a unique integer identifier and compiling the results into a static serialized word2id.json file.
- **label2id:** This system dictionary processes the comprehensive set of categorical entities, including project tags, skills, and hackathon interests, mapping them to a clean index space stored within a serialized label2id.json file.

For both index maps, any runtime token missing from these pre-compiled master arrays is automatically mapped to a standardized out-of-vocabulary ([UNK]) index slot. This fallback mechanism guarantees uniform matrix tensor dimensions and prevents downstream runtime embedding compilation failures during validation evaluations.

4. Tensor Generation

The final phase of the data preprocessing pipeline packages the synchronized integer sequences into model-ready parallel tensors,

enforcing strict structural alignment constraints across all input data streams:

- **Project Input Tensors:** Each project node is mapped using two simultaneous, aligned data streams. The first is a textual sequence tensor derived directly from project description. The second is a categorical tag tensor that captures the explicit technical requirements of the project.
- **Member Input Tensors:** Symmetrically matching the project configuration, each member node is structured into an input text stream tensor that combines personal biographies with aggregated project role descriptions. This is coupled with a parallel categorical skill tensor that reflects skills, and hackathon interests.

To enable efficient mini-batch operations, variable-length text sequences are standardized using uniform terminal padding and isolated via concurrent binary attention masks to prevent invalid data from corrupting the attention mechanisms.

• One-Hop Interaction Learning

Following the representation learning approach proposed in [5], the One-Hop module serves as the primary data-processing layer, generating initial latent representations by capturing localized textual, categorical, and structural identity tokens.

1. **Core Feature Initialization:** For N projects and M members, structural identities are mapped into continuous latent vector spaces via an integrated lookup table parameterized as:

$$e_{ID}^{\{(1)\}} = \left[\underbrace{v_{\{e_1\}}^{\{(1)\}}, \dots, v_{\{e_N\}}^{\{(1)\}}}_{task\ IDs}, \underbrace{u_{\{e_1\}}^{\{(1)\}}, \dots, u_{\{e_M\}}^{\{(1)\}}}_{developer\ IDs} \right] \quad (1)$$

2. **Multi-Head Self-Attention:** Free-form text data streams pass through a self-attention layer to learn context-aware token associations. For the k -th head, the relative importance score $a_{i,j}^k$ and updated word representation h_i^k between word i and word j are governed by trainable projection matrices w_1^k and w_2^k :

$$a_{\{i,j\}}^k = \frac{\{\exp(e_i^T w_1^k e_j)\}}{\sum_{\{m=0\}}^M \exp(e_i^T w_1^k e_m)} \quad (2)$$

$$h_i^k = w_2^k \left(\sum_{\{j=1\}}^M a_{\{i,j\}}^k e_j \right) \quad (3)$$

The individual head outputs are concatenated into a contextualized token vector: $h_i = [h_i^1; h_i^2; \dots; h_i^N]$

3. **Additive Attention Feature Pooling:** To compress variable-length token arrays, an additive attention pooling block calculates dynamic weights β_i^w using parameters a_w , b_w , and c_w to isolate high-priority terms:

$$\beta_i^w = \frac{\{\exp(a_w^{\top \tanh}(b_w \times h_i + c_w))\}}{\sum_{\{j=1\}}^n \exp(a_w^{\top \tanh}(b_w \times h_j + c_w))} \quad (4)$$

The unified textual summary vector is then compiled via weighted summation:

$$v_{desc} = \sum_{i=1}^n \beta_i^w h_i \quad (5)$$

4. **Multi-Stream Semantic Integration:** Processed text and label streams are combined. For projects, $v_t^{(1)} = [v_{desc}; v_{label}]$. The global one-hop semantic lookup index $e_t^{(1)}$ is organized as:

$$e_t^{(1)} = [v_{t1}^{(1)}, \dots, v_{tN}^{(1)}, u_{t1}^{(1)}, \dots, u_{tM}^{(1)}] \quad (6)$$

• Two-Hop Graph Learning

To capture deep collaborative patterns and overcome data sparsity constraints, this layer executes neighborhood message-passing operations across the bipartite topology, refining vectors via direct (first-order) and indirect (second-order) interactions.

1. **Neighborhood Attention Scoring Engine:** To filter noise, a parametric additive attention framework calculates localized structural coefficients across bipartite edges. For member neighbors ($m = |N_u|$) and project neighbors ($s = |N_v|$), normalized scores β_i^u and β_i^v are derived via independent trainable matrices:

$$\beta_i^u = \frac{\left\{ \exp \left(a_u^T \tanh \left(b_u \times u_{\{e_i\}}^{\{(2)\}} + c_u \right) \right) \right\}}{\sum_{\{j=1\}}^m \exp \left(a_u^T \tanh \left(b_u \times u_{\{e_j\}}^{\{(2)\}} + c_u \right) \right)} \quad (7)$$

$$\beta_i^v = \frac{\left\{ \exp \left(a_v^T \tanh \left(b_v \times v_{\{t_i\}}^{\{(2)\}} + c_v \right) \right) \right\}}{\sum_{\{j=1\}}^s \exp \left(a_v^T \tanh \left(b_v \times v_{\{t_j\}}^{\{(2)\}} + c_v \right) \right)} \quad (8)$$

2. **One-Hop (First-Order) Neighbor Aggregation:** Direct historical interactions are modeled via attention-weighted summation of immediate structural neighbors' features to compute localized context vectors $z_u^{(1)}$ and $z_v^{(1)}$:

$$z_u^{(1)} = \sum_{i \in \mathcal{N}_u} \beta_{ui} v_i^{(1)} \quad (9)$$

$$z_v^{(1)} = \sum_{j \in \mathcal{N}_v} \beta_{vj} u_j^{(1)} \quad (10)$$

3. **Two-Hop (Second-Order) Neighbor Aggregation:** The aggregation scope is extended to secondary paths ($\mathcal{N}_v$ of $\mathcal{N}_u$) to uncover implicit collaborative patterns like co-participation:

$$z_u^{(2)} = \sum_{k \in \mathcal{N}_u^{(2)}} \beta_{uk}^{(2)} u_k^{(1)} \quad (11)$$

$$z_v^{(2)} = \sum_{l \in \mathcal{N}_v^{(2)}} \beta_{vl}^{(2)} v_l^{(1)} \quad (12)$$

4. **Multi-Tier Representation Fusion:** To prevent over-smoothing, the system applies a dense concatenation operator ($\parallel$), blending localized base identity embeddings, direct historical connections, and indirect collaborative contexts into final descriptive vectors:

$$e_u = \left[u^{(1)} \parallel z_u^{(1)} \parallel z_u^{(2)} \right] \quad (13)$$

$$e_v = \left[v^{(1)} \parallel z_v^{(1)} \parallel z_v^{(2)} \right] \quad (14)$$

4.2.3 Bayesian Personalized Ranking Optimization

As detailed in the study “**Accuracy and Diversity-Aware Team Recommendation for Collaborative Software Development**” [5], to train the recommendation framework and optimize the parameter space of the neural modules, the framework implements the Bayesian Personalized Ranking (BPR) optimization model. Traditional regression metrics fail to capture the comparative preferences necessary for effective team formation. By contrast, BPR models learning as a pairwise ranking problem, maximizing the relative margin between a member’s observed historical collaborations and unobserved configurations.

- **Data Splitting and Pairwise Triplet Construction**

To evaluate model performance and safeguard against overfitting anomalies, the complete historical interaction dataset is partitioned using a Pareto-aligned 80/20 train-test split:

- **Training Sub-set (80%):** Utilized entirely by the optimization engine to compute gradients and update network parameters across epochs.
- **Validation/Testing Sub-set (20%):** Maintained in strict algorithmic isolation, reserved exclusively for relevance evaluation metrics.

The optimization process operates over structured interaction triplets rather than isolated individual data points. Let O^+ represent the set of all observed historical project-member interaction pairs (v, u) recorded in the platform's bipartite dataset. Symmetrically, let O^- represent the unobserved set of interactions, containing combinations (v, u') where member u' has no documented history of contribution to project v .

For every training iteration, a triplet $(v, u, u') \in O$ is dynamically generated such that:

$$O = \{(v, u, u') \mid (v, u) \in O^+ \wedge (v, u') \in O^-\} \quad (15)$$

This formulation forces the model to learn that project v exhibits a significantly higher structural affinity for historical teammate u than for randomly sampled candidate u' .

- **Mathematical Objective Function**

The system calculates a relevance prediction function $y(v, u)$ to measure the latent relevance between a project and a member. This score is derived by computing the continuous inner dot-product between their definitive, multi-hop representation vectors:

$$y(v, u) = \sigma(e_v^T \cdot e_u) \quad (16)$$

Where $\sigma(\cdot)$ represents the standard monotonic sigmoid activation function. To maximize the preference margins across all generated training pairs, the network architecture optimizes parameters by minimizing the formal BPR loss function $\mathcal{L}_{BPR}$:

$$\mathcal{L} = \sum_{(v,u,u') \in \mathcal{O}} -\ln \sigma(y(v, u) - y(v, u')) + \lambda \|\Theta\|_2^2 \quad (17)$$

Where the individual components of the objective function are defined as:

- $-\ln \sigma(\cdot)$ acts as the core optimization criteria, penalizing instances where the score of an unobserved candidate $y(v, u')$ approaches or exceeds the score of an active historical member $y(v, u)$.
- Θ represents the comprehensive set of all trainable parameters within the architecture, encompassing the identity lookup arrays e_{ID} , self-attention weight parameter matrices, and multi-hop neighborhood aggregation filters.
- $\lambda \|\Theta\|_2^2$ introduces an L^2 structural regularization penalty governed by the regularization coefficient λ . This mathematical constraint penalizes excessive weight magnitudes, ensuring the model remains generalizable and preventing parameter overfitting during optimization phases.

• Optimizer Dynamics

Model parameter updates are driven by the Adaptive Moment Estimation (Adam) optimization algorithm. Adam acts as an efficient backpropagation engine by dynamically computing individualized learning rates from first and second-order updates of the loss gradients. This adaptive scaling guarantees fast gradient convergence, protects the embedding space from local minima bottlenecks, and ensures stable numerical updates during training epochs.

Once the optimization phase converges, the stabilized weights are locked, allowing the system to output optimized representation matrices e_u and e_v to execute accurate real-time team recommendations.

• Hyperparameters

The following parameters are initialized to run the applied techniques (Semantic Encoding, Semantic Representation Learning, BPR):

Table 4.1: Sematic Representation Learning Module's Hyperparameters

Parameter Name	Symbol	Value	Description
Identity Embedding Dimension	d_{ID}	64	Initial width for unique structural member and project ID tokens.
Semantic Feature Dimension	d_{text}, d_{label}	128	Vector width for processing text sequences and label tags.
Core Representation Dimension	common_dim	256	The unified vector size for One-Hop, Two-Hop, and BPR scoring.
Batch Size	-	512	Scale of training triplet matrices processed per optimization step.
Learning Rate	η	$2e^{-4}$	Step size adjustment rate utilized by the Adam optimizer.
Regularization Weight	λ	$1e^{-5}$	L^2 penalty coefficient inside the BPR loss to prevent overfitting.
Dropout Rate	-	0.1	Regularization probability used to prevent neural overfitting.

4.3 Team Formation Engine - CsdRec Framework II

This section presents the team recommendation module that operates after semantic embeddings are generated and before evaluation. The module combines relevance-aware and diversity-aware optimization techniques to generate balanced hackathon teams, based on the framework proposed in the CsdRec paper [5].

The pipeline supports:

- Personalized team generation through pinned members.
- Relevance and diversity optimization.

- Candidate pool reduction for computational efficiency.
- Beam-search-based generation of multiple distinct teams.

● Hyperparameters

The following parameters are initialized before running the recommendation pipeline:

Table 4.2: Model Hyperparameters

Parameter	Value	Description
k	4	Team size — number of members per recommended team.
b	12	Bounding coefficient — candidate pool size = $b \times k$.
alpha	0.9	Balance factor between relevance and diversity.
K	12	Beam width — number of recommended teams generated by beam search.

The value of k is derived from the dataset using the median team size:

```
k = int(round(teams['members_count'].median()))
```

The selected value $\alpha = 0.9$ was chosen after experimenting with different relevance–diversity trade-offs. The final value provided the best balance between generating highly relevant teams while still maintaining sufficient diversity among team members.

In the experimental setup, k was initialized using the median team size in the dataset. In the deployed system, this parameter becomes dynamically configurable by the user.

4.3.1 Relevance Awareness

• Individual Member Relevance — $y(v, u)$

Measures how relevant a member u is to a target project v using semantic embeddings.

Based on Equation (18) [5]:

$$y(v, u) = \sigma(e_v^T e_u) \quad (18)$$

Where:

- e_v : project embedding.
- e_u : member embedding.
- σ : sigmoid activation function.

The dot product is scaled by 16 before applying sigmoid to avoid score saturation in the 256-dimensional embedding space.

```
def y(project_emb, member_emb):  
    dot_prod = np.dot(project_emb, member_emb)  
    return sigmoid(dot_prod / 16.0)
```

Output values range between (0,1), where higher values indicate stronger relevance.

• Team Relevance — $s(v, A_k)$

Measures the average relevance of all members inside team A_k .

$$s(v, A_k) = \frac{1}{k} \sum_{u \in A_k} y(v, u) \quad (19)$$

```
def s_relevance(team_indices, score_dict):  
    if not team_indices:  
        return 0.0  
    return sum(score_dict.get(i, 0.0) for i in team_indices) /  
    len(team_indices)
```

All individual relevance scores are precomputed once and stored inside `score_dict` to avoid repeated calculations during beam search.

4.3.2 Skill Diversity Awareness

• Pairwise Similarity — $\text{sim}(u_i, u_j)$

Measures similarity between two members using their embeddings.

$$\text{sim}(u_i, u_j) = \sigma(e_{u_i}^T e_{u_j}) \quad (20)$$

```
def sim(u_emb1, u_emb2):  
    dot_prod = np.dot(u_emb1, u_emb2)  
    return sigmoid(dot_prod / 16.0)
```

- **Team Diversity — $g(A_k)$**

Calculates team diversity based on pairwise dissimilarity between members.

$$g(A_k) = 1 - \frac{2}{k(k-1)} \sum_i \sum_{j < i} \text{sim}(u_i, u_j) \quad (21)$$

```
def g_diversity(team_members_indices, emb_dev):
```

Values close to:

- 1 → highly diverse teams.
- 0 → highly similar teams.

When team size = 1, diversity defaults to 0 because diversity cannot be computed for a single member.

- **Compatibility Function**

The compatibility function combines relevance and diversity into a unified optimization objective.

$$f(v, A_k) = \alpha \cdot s(v, A_k) + (1 - \alpha) \cdot g(A_k) \quad (22)$$

With:

- $\alpha = 0.9$
- Strong emphasis on relevance while preserving diversity.

```
def compatability_score(team_members_indices,
                        score_dict,
                        emb_dev,
                        alpha):
```

This function represents the primary objective optimized during team generation.

4.3.3 Members Matching Algorithm

- **Candidate Pool Construction**

Instead of searching across all available users, the system first reduces the search space by selecting only the most relevant candidates.

The candidate pool size is:

$$|A| = b \times k \quad (23)$$

Steps:

- Compute relevance scores for all members.
- Sort members by relevance score.
- Select the top $b \times k$ members.
- Add pinned members if they are not already included.

```
def build_candidate_set_A(...):
```

The updated implementation uses vectorized matrix operations to compute relevance scores efficiently for all members simultaneously.

• **Personalized Team Formation using Pinned Members**

The updated system supports personalized team generation by allowing users to pre-select specific members before recommendation begins.

These members are treated as fixed team members during beam search.

1. Pinned Members Initialization

If the user provides predefined team members:

- They are inserted into the initial beam.
- They remain fixed throughout the recommendation process.
- Only remaining positions are optimized by the algorithm.

```
pinned_members = list(pinned) if pinned else []
```

The system validates that the number of pinned members does not exceed the target team size.

2. Candidate Pool Adjustment

Pinned members are removed from the candidate pool to prevent duplication during generation.

Additionally, pinned members are forcefully inserted into the candidate pool if they were excluded by relevance filtering.

3. Remaining Slot Optimization

Instead of generating a full team from scratch, the algorithm fills only the remaining positions:

```
remaining = team_size - len(pinned_members)
```

If pinned members already fill the entire team:

→ The algorithm immediately returns the predefined team.

• Bounded Greedy Beam Search

Building upon the generation framework presented in [5], this module implements an extended bounded greedy beam search to overcome the limitations of single-path selection and generate optimized, diverse project teams.

1. Greedy Selection Objective

The recommendation process iteratively expands candidate teams by selecting the member that maximizes the compatibility score.

Based on Equation (24) [5]:

$$A_{i+1} = A_i \cup \{\arg \max f(v, A_i \cup \{u\})\} \quad (24)$$

2. Beam Search Extension

The implementation extends the original greedy algorithm by maintaining multiple partial solutions simultaneously.

Steps:

- Initialize beam with pinned members or an empty team.
- Expand each partial team using candidate members.
- Compute relevance, diversity, and compatibility scores.
- Keep only the top K candidate teams after each iteration.
- Continue until all remaining positions are filled.
- Duplicate Team Prevention

The updated implementation prevents duplicate teams by using canonical sorted team representations:

```
team_key = tuple(sorted(new_team))
```

Only the highest-scoring version of each unique team is preserved during beam pruning.

3. Member ID Mapping

Beam search operates using internal embedding indices. After recommendation, teams are converted back into actual member IDs.

```
topK_actual_ids = [  
    [member_ids_list[idx] for idx in team]  
    for team in topK_teams_indices]
```


4.4 Dynamic Runtime Configuration

During the experimental and evaluation phase, several hyperparameters such as team size (k), members and project embeddings were initialized using fixed values derived from the dataset in order to ensure consistent evaluation and fair comparison across experiments.

However, in the deployed web application, the recommendation pipeline was extended to support dynamic runtime configuration based on user input. Instead of relying on a fixed team size extracted from the dataset, users can specify their desired team size during recommendation generation. Additionally, project information, required skills, preferences, and pinned members are provided dynamically through the user interface.

Despite these runtime adaptations, the underlying recommendation methodology, compatibility scoring, graph-based representation learning, and greedy beam search pipeline remain unchanged. The system therefore preserves the same recommendation logic while offering greater flexibility and personalization for real-world hackathon team formation scenarios.

4.5 Technologies Used

Table 4.3: Technologies Used in Team Formation Module

Technology / Library	Description / Role
Python	Core programming language used to implement the recommendation pipeline, graph-based learning modules, and team generation algorithms.
Pandas (import pandas as pd)	Used for loading, preprocessing, filtering, and managing structured datasets containing users, projects, skills, and interactions.
NumPy (import numpy as np)	Used for numerical computations, vector operations, embedding manipulation, similarity calculations, and matrix processing.
PyTorch (import torch)	Deep learning framework used to implement embedding learning, attention mechanisms, graph-based modeling, and neural recommendation components.
Torch Geometric (PyG)	Used for graph-based learning operations and implementing one-hop and two-hop neighborhood

Technology / Library	Description / Role
	aggregation within the collaboration graph.
Scikit-learn (sklearn)	Used for preprocessing utilities, similarity computations, and evaluation-related operations.
Natural Language Processing (NLP) Techniques	Used for preprocessing textual data such as user skills, project descriptions, and labels through tokenization, normalization, and semantic preparation.
Multi-Head Self-Attention	Used to capture contextual relationships between words and labels while generating semantic embeddings.
Additive Attention Pooling	Used to assign importance weights to tokens and aggregate informative semantic features into unified representations.
Semantic Embeddings	Used to represent users and projects as dense vector representations that capture semantic similarity and compatibility.
Bipartite Graph Construction	Used to model historical collaboration relationships between users and projects as graph structures.
One-Hop Graph Learning	Used to capture direct relationships between neighboring users and projects within the collaboration graph.
Two-Hop Graph Learning	Used to incorporate indirect graph relationships and contextual collaboration patterns into refined embeddings.
Bayesian Personalized Ranking (BPR) Loss	Ranking optimization technique used to improve recommendation quality by prioritizing relevant user–project matches over irrelevant ones.
Greedy Beam Search Algorithm	Used to iteratively generate Top-K team recommendations while optimizing relevance and diversity simultaneously.
Candidate Pool Construction	Used to reduce the recommendation search space by selecting the most relevant candidate members before beam search execution.
Diversity-Aware Optimization	Used to ensure generated teams maintain balanced skill diversity and reduced member similarity.
FastAPI	Used to integrate AI modules with the backend API

Technology / Library	Description / Role
	and expose recommendation services to the web application.

4.6 Hardware & Software Configuration

To develop, run, and test the Team formation Module, the following hardware and software configurations were used:

Hardware Configuration

Development Machine:

- **Processor:** AMD Ryzen 7 PRO 3700U w/ Radeon Vega Mobile Gfx (2.30 GHz)
- **RAM:** 16 GB
- **Storage:** 323 GB
- **Graphics Card:** AMD Radeon (TM) RX Vega 10 Graphics (2 GB)
- **Operating System:** Windows 11 Pro 64-bit
- **System Type:** x64-based processor

Cloud Environment:

- **Platform:** Google Colab (for model execution, clustering, and optimization tasks)
- **Instance Type:** Standard Colab Runtime
- **CPU:** 2-core Intel Xeon (shared)
- **RAM:** Approx. 12.72 GB

Software Configuration

- **Programming Language:** Python 3.10
- **Development Tools:**
- **Jupyter Notebook / Google Colab / Kaggle:** for scripting and testing
- **Visual Studio Code** for local code development and debugging.
- These configurations ensured smooth data handling, recommendation generation, clustering, and route.

4.7 Experiments and Results

To evaluate the effectiveness of the proposed recommendation pipeline, a series of experiments were conducted on the collected Devpost dataset after completing the embedding generation and team recommendation stages. The evaluation focused on measuring both recommendation accuracy and team diversity, which

represent the two primary objectives of the proposed system.

The experiments were performed on approximately 8,400 projects, where the greedy beam search algorithm generated Top- K recommended teams with K values ranging from 1 to 12. The evaluation process followed the same methodology and metrics adopted in the CsdRec framework in order to ensure fair comparison with the original research paper [5].

The recommendation pipeline was evaluated using the following metrics:

- **Mean Precision (MP):** Measures how accurately the system recommends relevant team members for a project.
- **Mean Recall (MR):** Measures the system's ability to retrieve relevant members from the available candidate pool.
- **Mean Inter-List Diversity (MILD):** Measures the diversity between generated recommendation lists to ensure variation and reduced similarity among recommended teams.

The obtained results demonstrated that the proposed system achieved strong performance across all evaluation metrics.

Table 4.4: Results

Evaluation Metric	CSDRec (Kaggle)	CSDRec (GitLab)	Team Catalyst (Devpost)
MP	0.5890	0.6624	0.6884
MR	0.7243	0.7926	0.9572
MILD	0.6061	0.5878	0.6038

The results indicate that the proposed Team Catalyst pipeline achieved a **Mean Precision score exceeding 68%**, outperforming the results reported in the original CsdRec paper on both the Kaggle and GitLab datasets. This demonstrates the system's strong capability in recommending highly relevant team members for hackathon projects.

In terms of **Mean Recall**, the proposed model achieved a score **exceeding 95%**, showing a significant improvement over the results presented in the original paper. This indicates that the system was highly effective in retrieving relevant and compatible members from the candidate pool.

For **Mean Inter-List Diversity (MILD)**, the model achieved a score **exceeding 60%**, outperforming the GitLab-based results from the paper and achieving performance nearly equivalent to the Kaggle benchmark. This confirms the system's ability to maintain diversity among generated teams while preserving recommendation quality. [5]

• Performance Visualization

Furthermore, the performance of the proposed recommendation pipeline is illustrated in Figure 4.1, which evaluates the system across the three primary metrics (MP, MR, and MILD). The plotted line charts demonstrate how these evaluation metrics behave as the algorithm generates a varying number of recommended teams K ranging from 1 to 12, providing a clear visualization of the algorithmic trade-offs between recommendation accuracy and team diversity.

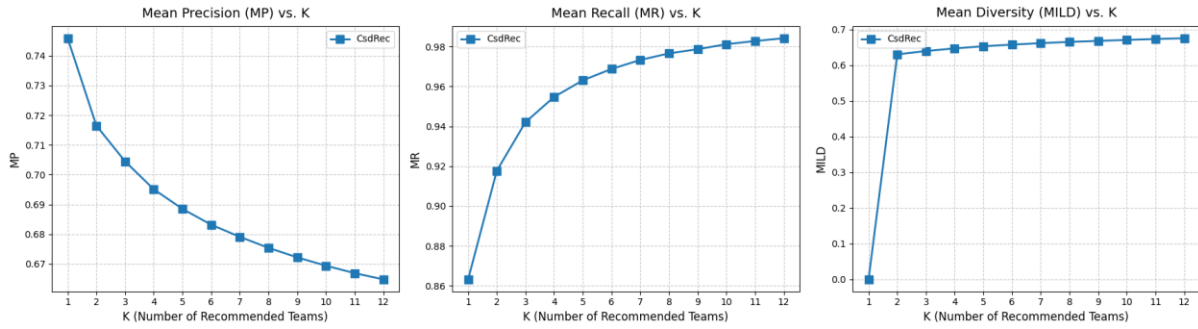


Figure 4.1 :performance visualization

Overall, the experimental results confirm that the proposed recommendation pipeline successfully achieved the two primary objectives of the project:

1. Generating highly relevant and compatible teams.
2. Maintaining diversity and balanced team composition.

Furthermore, the obtained results demonstrate that the proposed implementation not only successfully adapted the CsdRec framework to the hackathon team formation domain, but also achieved performance improvements over the original research paper in several evaluation aspects.

Chapter 5: Run the Application

5.1 Entry Page (Landing Page).....	51
5.2 User Registration	52
5.3 User Login.....	53
5.4 Password Reset.....	53
5.5 Home Page	55
5.6 Profile Completion	56
5.7 Create Team Request	57
5.8 Project Ideas Exploration.....	60
5.9 Hackathons Exploration	61
5.10 Interested Hackathons	62
5.11 Team Recommendation.....	62
5.12 Team Progress Track.....	64
5.13 Team Invitations	64
5.14 Tracking Team Progress as an Invited Member.....	65
5.15 Viewing Participant Profiles.....	66
5.16 Team Rematching.....	67
5.17 User Dashboard	69
5.18 Admin Dashboard.....	69

Chapter 5: Run the Application

Welcome to the user manual for **TeamCatalyst**, a web platform designed to simplify hackathon team formation using artificial intelligence. This manual provides a step-by-step guide to the platform's main features, including creating team requests, submitting project ideas, joining hackathons, receiving AI-powered team recommendations, and managing your teams. By following this guide, you will learn how to use the platform effectively to find compatible teammates and build successful hackathon teams.

5.1 Entry Page (Landing Page)

- The **landing page** is the main entry point to the platform. It introduces users to the platform's purpose
- The page provides a brief overview of the platform's main features and show that users can create teams or join hackathons to find suitable teammates. It's simple and intuitive design helps users quickly understand the platform and encourages them to sign up or log in to get started.

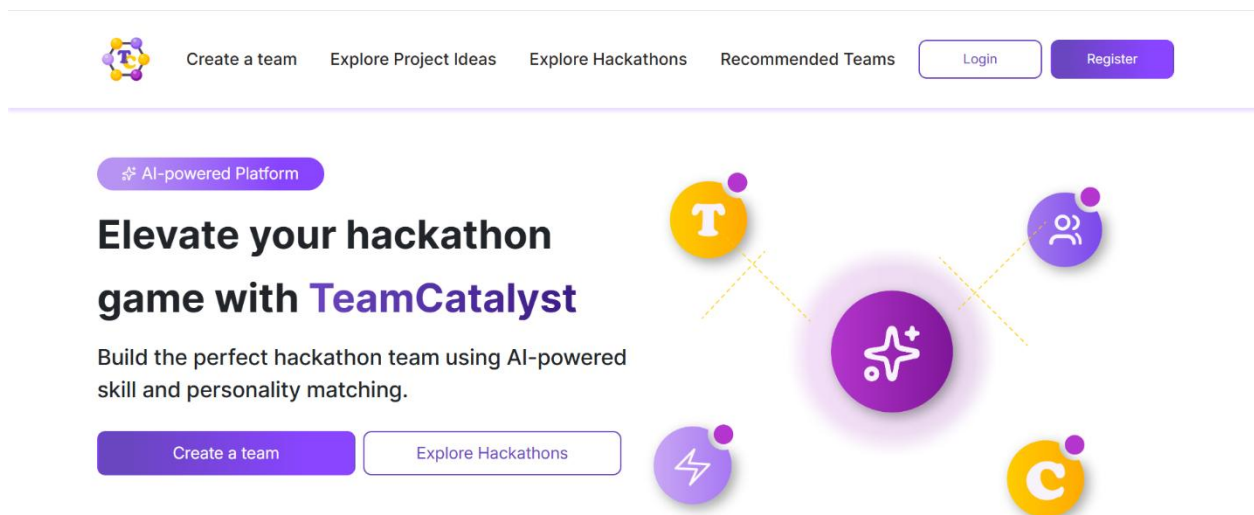
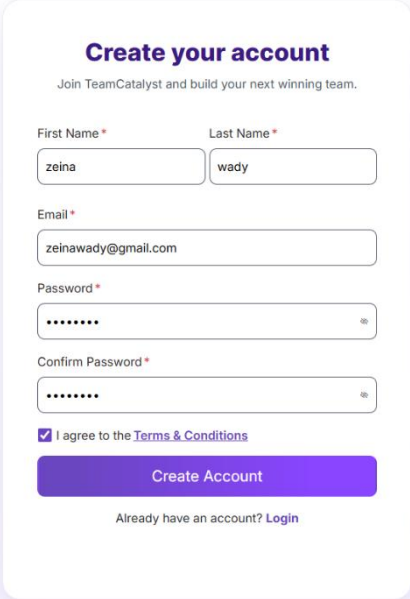


Figure 5.1: Entry Page (Landing Page)

5.2 User Registration

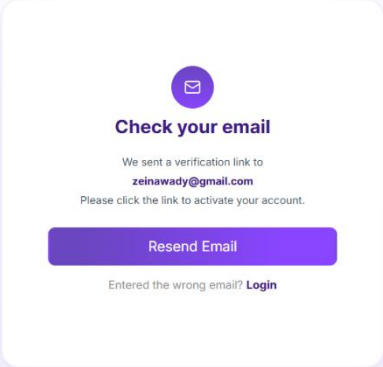
- The registration page allows new users to create an account and access the platform's features. It can be accessed by clicking the **Sign-Up** button located in the navigation bar.
- The page contains a simple registration form where users are required to enter **their first name, last name, email address, and password**. After filling in the required information, users can submit the form to begin the registration process.



The registration form is titled "Create your account" in bold purple text. Below the title is a subtitle: "Join TeamCatalyst and build your next winning team." The form contains several input fields: "First Name *" with the value "zeina", "Last Name *" with the value "wady", "Email *" with the value "zeinawady@gmail.com", "Password *" with masked characters "*****", and "Confirm Password *" also with masked characters "*****". There is a checkbox labeled "I agree to the Terms & Conditions" which is checked. Below the inputs is a large purple button labeled "Create Account". At the bottom, there is a link: "Already have an account? Login".

Figure 5.2 : User Registration

- Once the registration form has been successfully submitted, a **verification email** is automatically sent to the email address provided. Users should open the email and click the verification link to validate their email address.



The screen is titled "Check your email" in bold purple text, preceded by a purple envelope icon. Below the title, it says: "We sent a verification link to zeinawady@gmail.com" and "Please click the link to activate your account." There is a large purple button labeled "Resend Email". At the bottom, there is a link: "Entered the wrong email? Login".

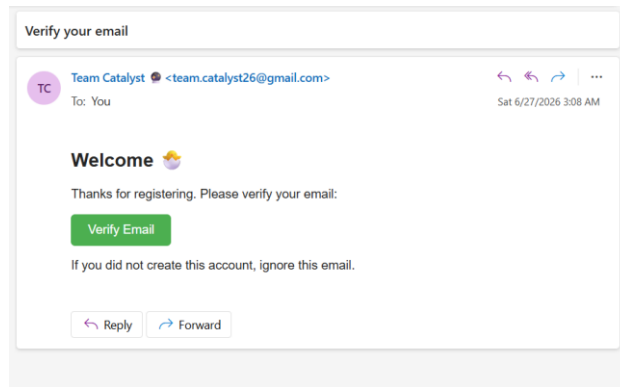


Figure 5.3: Email Verification

5.3 User Login

- Upon successful verification, the account is activated and the user can sign in and use the platform.

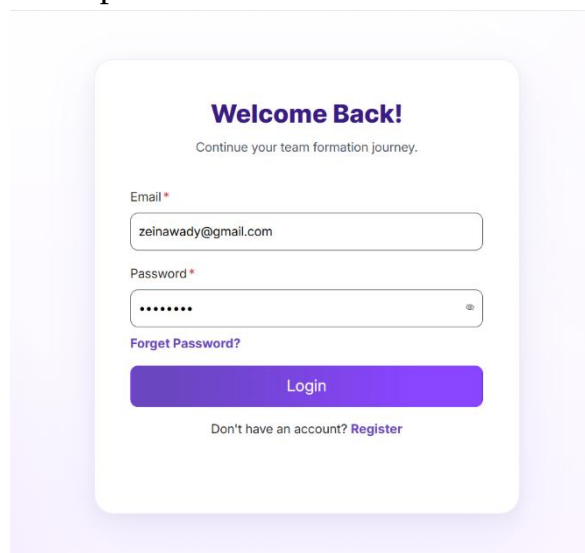


Figure 5.4: User Login

5.4 Password Reset

- The Forgot Password page allows users to reset their password if they are unable to access their account. Users can access this page by clicking the Forgot Password link on the login page.
- A password reset screen is displayed, prompting the user to enter the email address associated with their account.

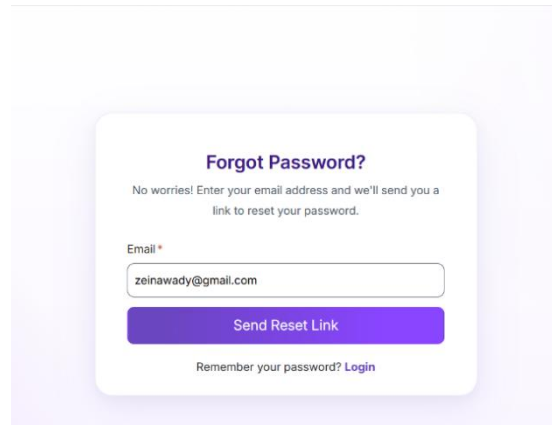
A web form titled "Forgot Password?" with a subtitle "No worries! Enter your email address and we'll send you a link to reset your password." It features an "Email *" input field containing "zeinawady@gmail.com", a purple "Send Reset Link" button, and a "Remember your password? Login" link at the bottom.

Figure 5.5: Password Reset - 1

- After the user submits their email address, a password reset email is automatically sent to that email. The email contains a secure link that directs the user to the password reset page, where they can create and confirm a new password. Once the new password is successfully saved, the user can sign in to the platform using their updated credentials.

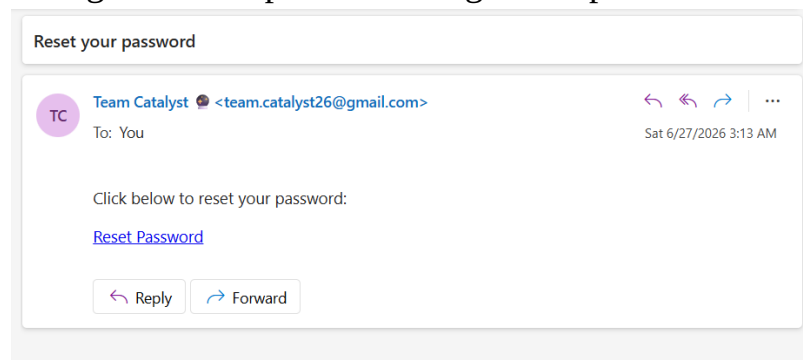
An email interface showing a message from "Team Catalyst" with the email address "<team.catalyst26@gmail.com>". The subject is "Reset your password" and the recipient is "To: You". The email body says "Click below to reset your password:" followed by a blue "Reset Password" link. At the bottom are "Reply" and "Forward" buttons. The timestamp is "Sat 6/27/2026 3:13 AM".

Figure 5.6: Password Reset - 2

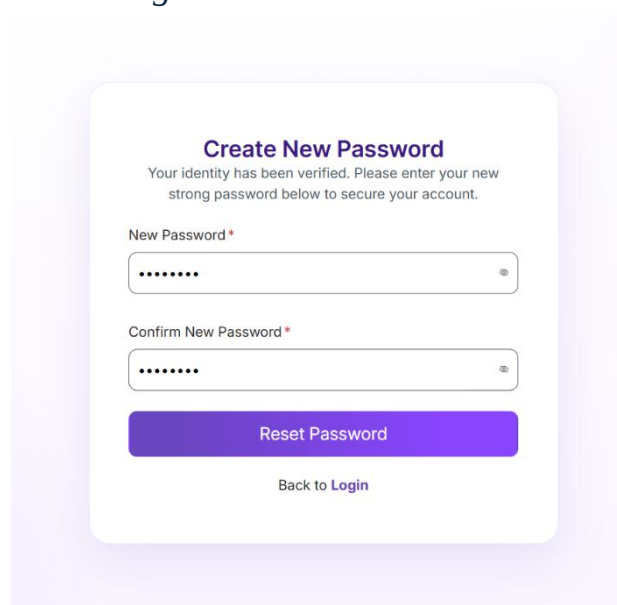
A web form titled "Create New Password" with a subtitle "Your identity has been verified. Please enter your new strong password below to secure your account." It features two password input fields: "New Password *" and "Confirm New Password *", both masked with dots. Below the fields is a purple "Reset Password" button and a "Back to Login" link at the bottom.

Figure 5.7: Password Reset - 3

5.5 Home Page

- The Home page is designed to provide users with a clear introduction to the platform and an engaging browsing experience. It consists of several well-structured sections that explain the platform's purpose and guide users through its main functionalities.

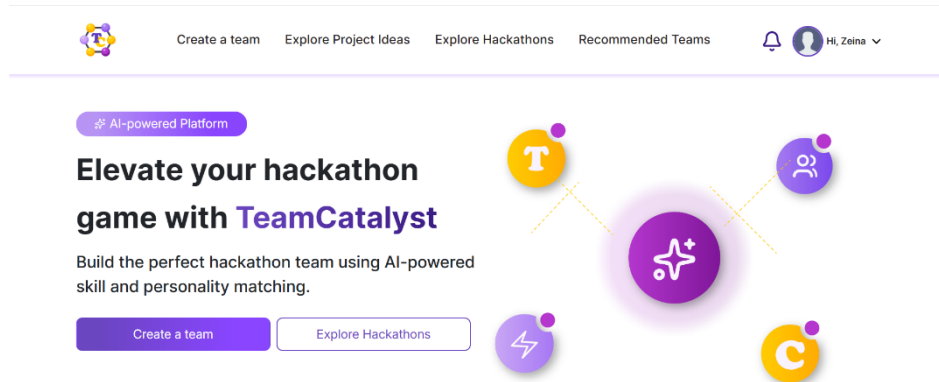
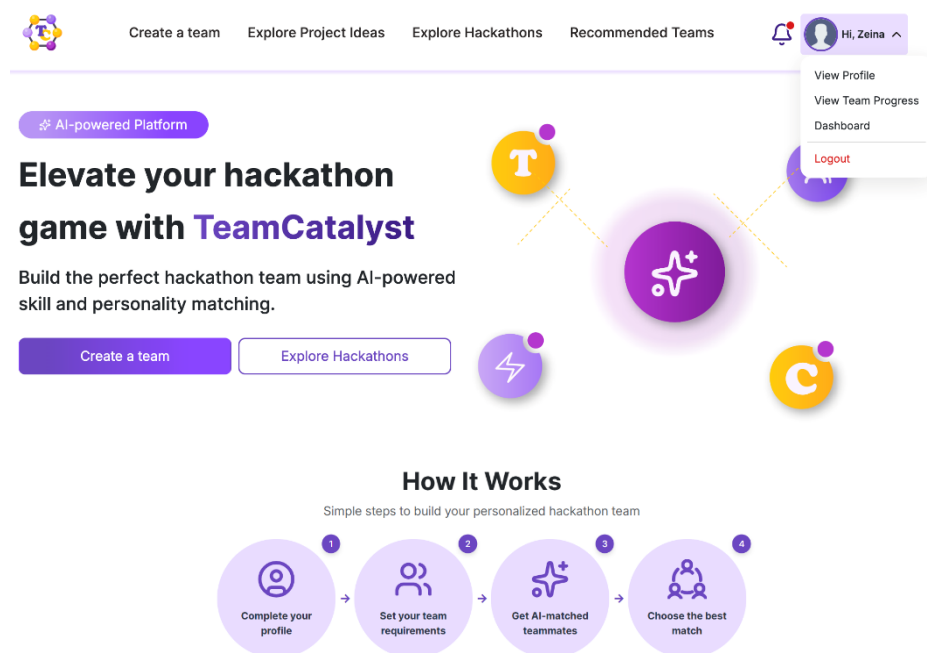


Figure 5.8: Home Page - 1



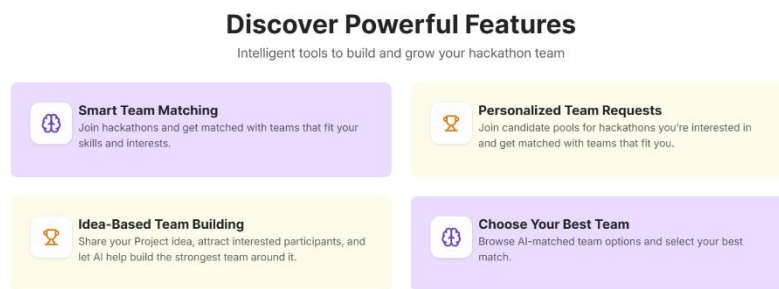


Figure 5.9: Home page - 2

- A Frequently Asked Questions (FAQ) section is also included to answer common user inquiries and provide a better understanding of how the platform works.

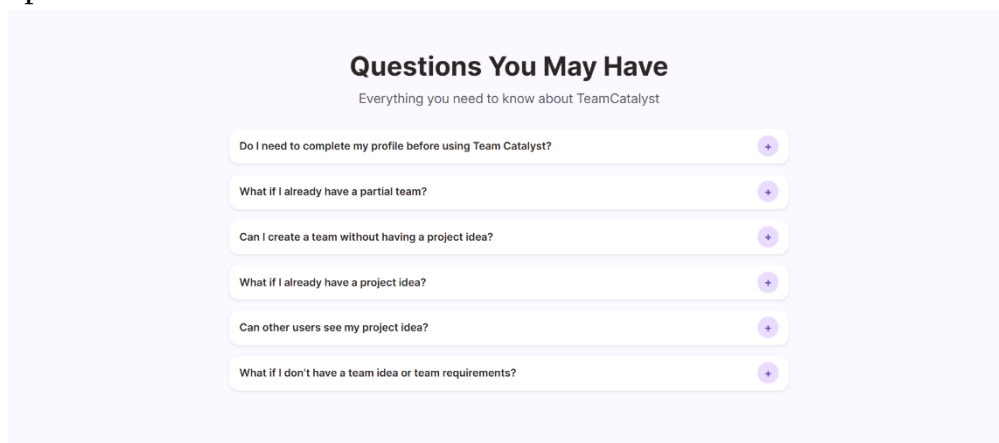


Figure 5.10: Home Page - 3

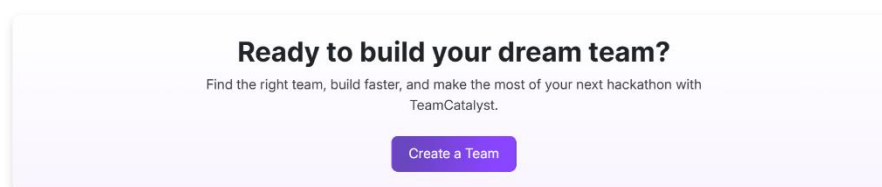


Figure 5.11: Home page - 4

5.6 Profile Completion

- Before accessing the platform's team formation features, **users are required to complete their profile**. A complete and accurate profile is essential because it provides the information used by the AI model to recommend suitable teammates and build compatible teams.
- The profile completion page allows users to upload a profile picture and

provide personal and professional information, including a short bio, email address, GitHub account, LinkedIn account, primary roles, and skills. Among these, the **roles** and **skills** are the most important, as they are the primary factors the AI model uses to evaluate compatibility and create well-balanced hackathon teams. Therefore, users are encouraged to provide accurate and up-to-date information to achieve the best possible matching results.

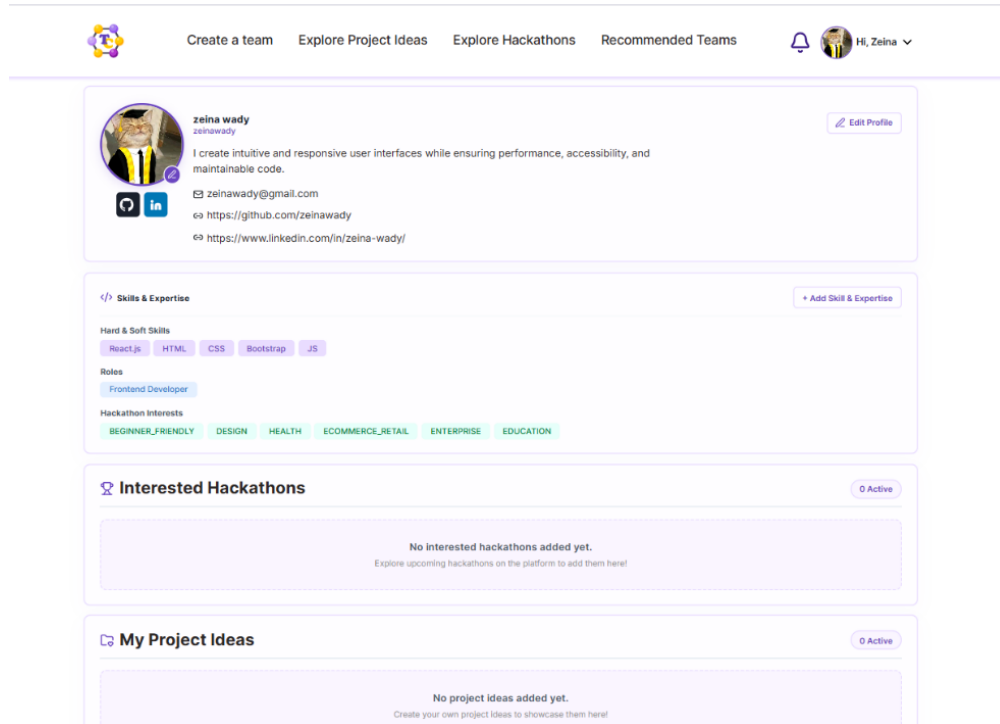


Figure 5.12: Profile Completion

5.7 Create Team Request

1. The **Create Team Request** page allows users to define the team they want to build for a specific hackathon. Users begin by entering the team's basic information, including the **team's name** and the **hackathon** for which the team is being created. Users can optionally provide a **project idea** by entering its description. Users also specify the **number of teammates needed**, along with the **required roles** and **technical skills**. This information serves as the primary input for the AI model, enabling it to recommend compatible teammates who best match the team's requirements.

Create Your Dream Team
Build an amazing team for your next hackathon adventure

Team Basics Invite Members Required Skills Final Details

Team Basics

Team Name
AI Resume Screening Platform

Hackathon Name *
Backend Innovation Challenge 2026

☒ I already have a project idea for this hackathon

Project Description *
An AI-powered platform that analyzes resumes, ranks candidates based on job descriptions, and provides personalized feedback to improve resumes.
144/500 characters

Required Team Members *
2 3 4 5 Other

Next Step

Quick Stats

- Team Size: 0/4
- Skills: 0
- Roles: 0
- Completion: 1/4

Pro Tips

- Choose a team name that's memorable and represents your project
- Required Team Members only includes the teammates you're looking for, not yourself.
- Diverse skills lead to stronger teams - mix technical and creative talents
- Clear project descriptions attract the right team members

Figure 5.13: Create Team Request - step 1

- The platform also allows users to invite a specific person to join their team. To do this, the user enters the invited member's email address while creating the team request. An invitation is then sent to the specified user. If the invitation is accepted, that member is treated as a **fixed team member** and is automatically included in every AI-generated team recommendation. The AI model then recommends the remaining teammates while keeping the accepted member as part of the final team composition.

Create Your Dream Team
Build an amazing team for your next hackathon adventure

Team Basics Invite Members Required Skills Final Details

Build Your Team

Team Progress: 0/4
0% complete

Search by name or email... Invite

zeina wady Member

Previous Next Step

Quick Stats

- Team Size: 0/4
- Skills: 0
- Roles: 0
- Completion: 2/4

Pro Tips

- Choose a team name that's memorable and represents your project
- Required Team Members only includes the teammates you're looking for, not yourself.
- Diverse skills lead to stronger teams - mix technical and creative talents
- Clear project descriptions attract the right team members

Figure 5.14: Create Team Request - step 2

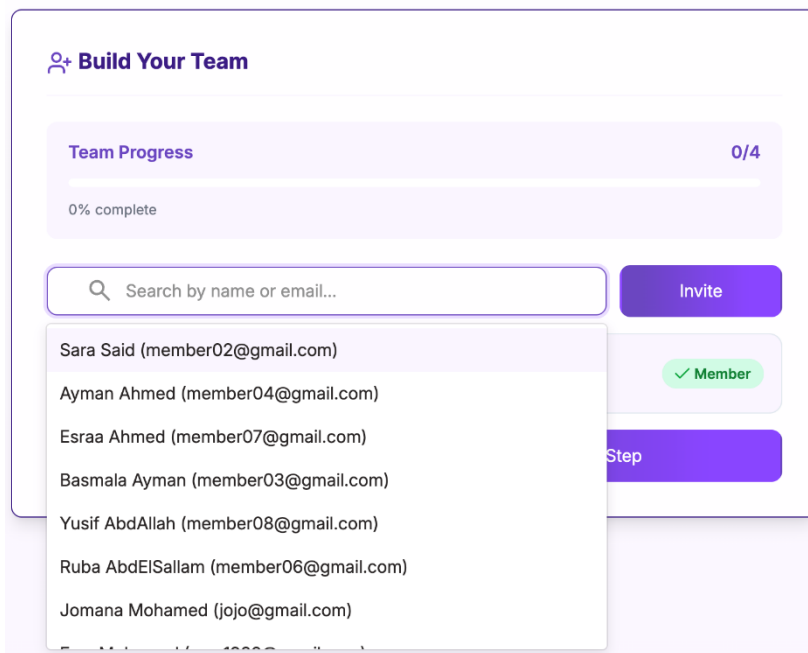


Figure 5.15: Create Team Request – Invite Members

- After providing the team's basic information, the user specifies the **required roles** and skills. These requirements serve as the primary criteria used by the AI model to recommend suitable teammates.

To achieve the best matching results, users should define the roles and skills accurately and include all the expertise required for the project. Providing clear and diverse requirements enables the AI model to build a well-balanced team.

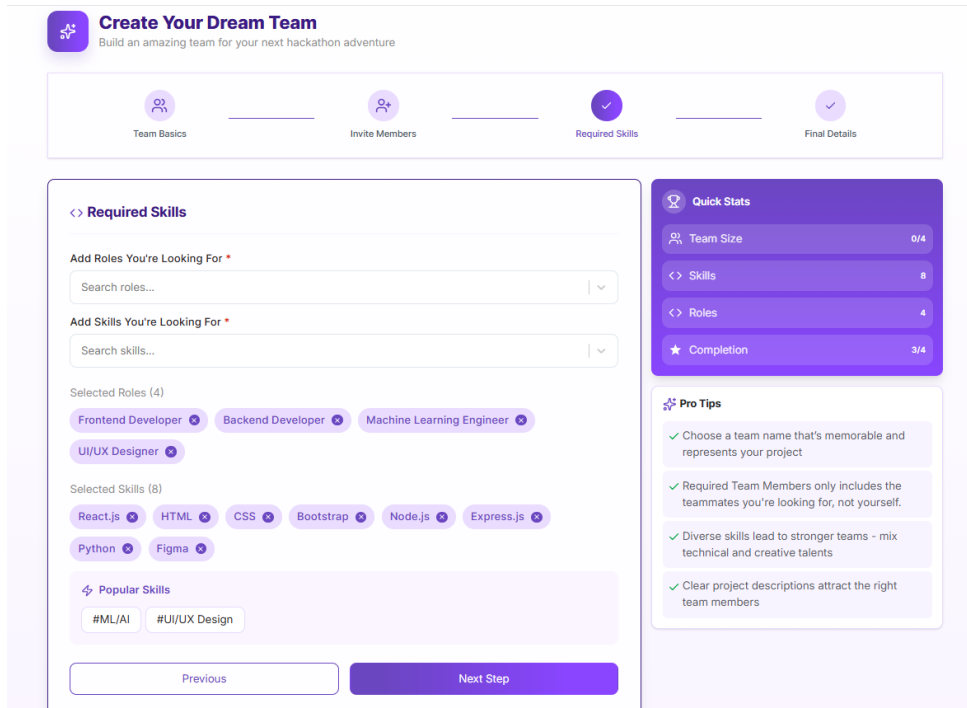


Figure 5.16: Create Team Request - step 3

- Finally, a summary of the team request is displayed, allowing the user to

review all the information entered before submitting it.

Create Your Dream Team
Build an amazing team for your next hackathon adventure

Team Basics | Invite Members | Required Skills | **Final Details**

Final Details

Team Summary

- Team Name: AI Resume Screening Platform
- Hackathon Name: Backend Innovation Challenge 2026
- Team Members: 0 member
- Required Skills: 8 skills
- Required Roles: 4 Roles

Previous | **Create Team**

Quick Stats

- Team Size: 0/4
- Skills: 8
- Roles: 4
- Completion: 4/4

Pro Tips

- ✓ Choose a team name that's memorable and represents your project
- ✓ Required Team Members only includes the teammates you're looking for, not yourself.
- ✓ Diverse skills lead to stronger teams - mix technical and creative talents
- ✓ Clear project descriptions attract the right team members

Figure 5.17: Create Team Request - step 4

- Once the user confirms that all the information is correct, the team request is submitted successfully. The request then remains active until enough users are available for the AI model to generate reliable team recommendations.

Create Your Dream Team
Build an amazing team for your next hackathon adventure

Team Basics | Invite Members | Required Skills | **Final Details**

Stay Tuned!

We're building your team "AI Resume Screening Platform" now! Hang tight

AI is Matching Your Team

Our intelligent system is finding the perfect teammates based on your skills, interests, and project requirements.

✓ You'll receive a notification when matches are ready

Got It

Figure 5.18: Create Team Request - step 5

5.8 Project Ideas Exploration

- If a user provides a project idea while creating a team request, it is automatically published in the **Project Ideas** Page. This section displays all project ideas associated with different hackathons, allowing users to explore ideas that match their interests.

Discover Project Ideas

Browse exciting projects and join teams that match your skills



zeina wady
Project Creator

AI Resume Screening Platform

An AI-powered platform that analyzes resumes, ranks candidates based on job descriptions, and provides personalized feedback to improve resumes.

Backend Innovation Challenge 2026 46 min ago

Required Roles

#frontend #backend #machine-learning +1 more

Required Skills

#react-js #html #css +5 more

Show Full Project Details ...

Team 0/4 0 interested

I'm Interested

Figure 5.19: Project Ideas Exploration

5.9 Hackathons Exploration

- The **Hackathons** page displays all hackathons available on the platform, allowing users to explore and choose the hackathons that match their interests.
- By applying to a hackathon, users indicate that they are interested in participating in that event. They are then considered by the AI model when generating teams for general team requests created for the same hackathon. When a team owner submits a request without a specific project idea, the AI model selects suitable teammates from the users who have applied to that hackathon.

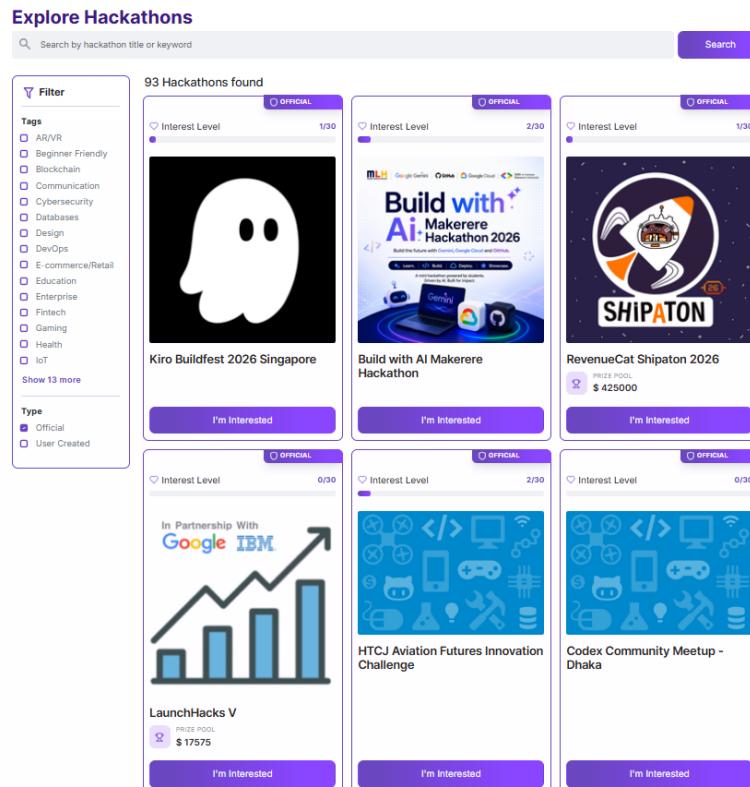


Figure 5.20: Hackathons Exploration

5.10 Interested Hackathons

- The Interested Hackathons section in the user's profile displays all the hackathons that the user has expressed interest in. This allows users to easily keep track of the hackathons they are participating in.

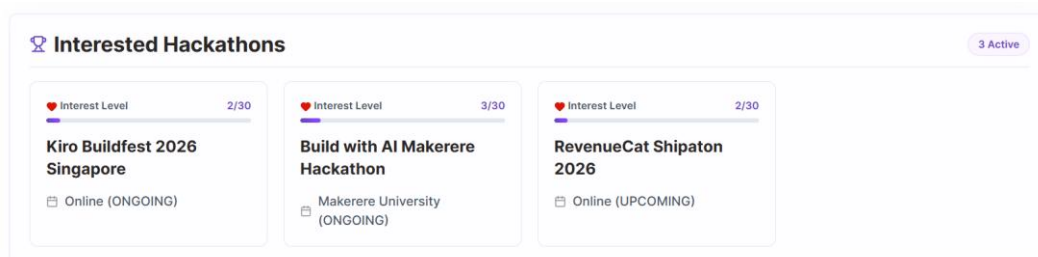


Figure 5.21: Interested Hackathons

5.11 Team Recommendation

- Once the AI model has generated the team recommendations, the user is notified through the platform.

Notifications

Stay updated with your teams and hackathons

Filter

All Notifications

Team Invitations

Team Updates

Accepted

Rejected

Recommendations

Matches

All1 unread

Mark all as read

Your team recommendations are ready

RECOMMENDATION RECEIVED

We found 3 recommended team(s) for "Backend Innovation Challenge 2026". Check your recommendations page.

Jun 27, 03:54 AM

View Recommended Teams

Figure 5.22: Team Recommendation - Notification

- Once the recommendations are ready, the platform displays multiple team options generated by the AI model. The user can review the recommended members in each option, compare the available teams, and select the team combination that best meets their needs.

Recommended Teams

Find the perfect team to join and collaborate on exciting hackathon projects

My TeamsInvitations

Member Recommendations for Team AI Resume Screening Platform:3 Options

Option 1

Team Members

Basmala AymanFull stack developer

Sherif OmarProduct manager

Fathy IbrahimData scientist

Wael MagdyML engineer

Accept TeamReject

Option 2

Team Members

Tarek FaroukBackend developer

Sherif OmarProduct manager

Sama ElKenawyFrontend developer

Wael MagdyML engineer

Accept TeamReject

Option 3

Team Members

Basmala AymanFull stack developer

Sherif OmarProduct manager

Mohamed KarimFull stack developer

Wael MagdyML engineer

Accept TeamReject

Figure 5.23: Team Recommendation - All Recommended Teams' Options

5.12 Team Progress Track

- After selecting a recommended team, the user can monitor its progress through the Team Progress page. This page displays all members of the selected team along with the current status of their invitations.
- For each team member, the user can view whether the invitation has been accepted, rejected, or is still pending. This allows the user to track the team formation process in real time and determine when the team is ready to be finalized or whether additional teammates need to be recommended.

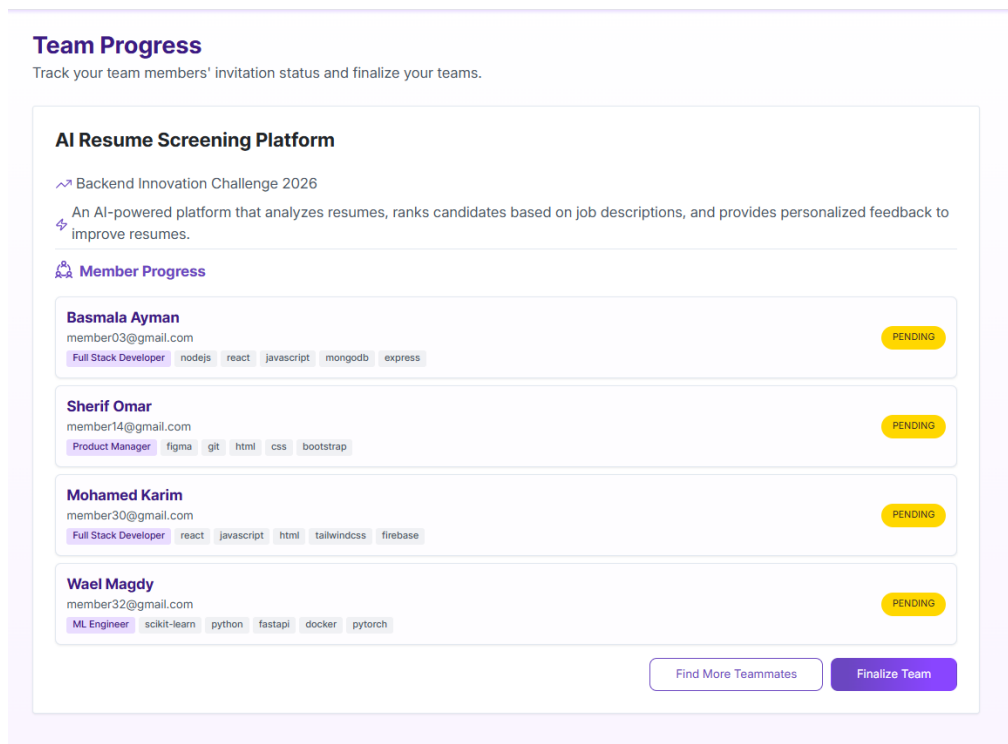


Figure 5.24: Team Progress Track

5.13 Team Invitations

- Users who are included in a recommended team receive a notification of team invitation.

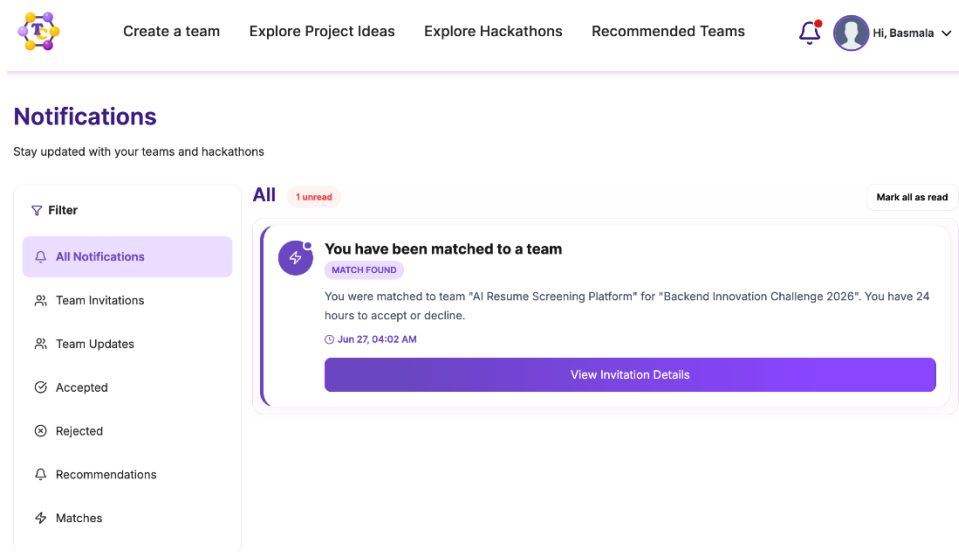


Figure 5.25: Team Invitations - Notifications

- The invitation contains the team details and allows the invited user to review the request before making a decision.

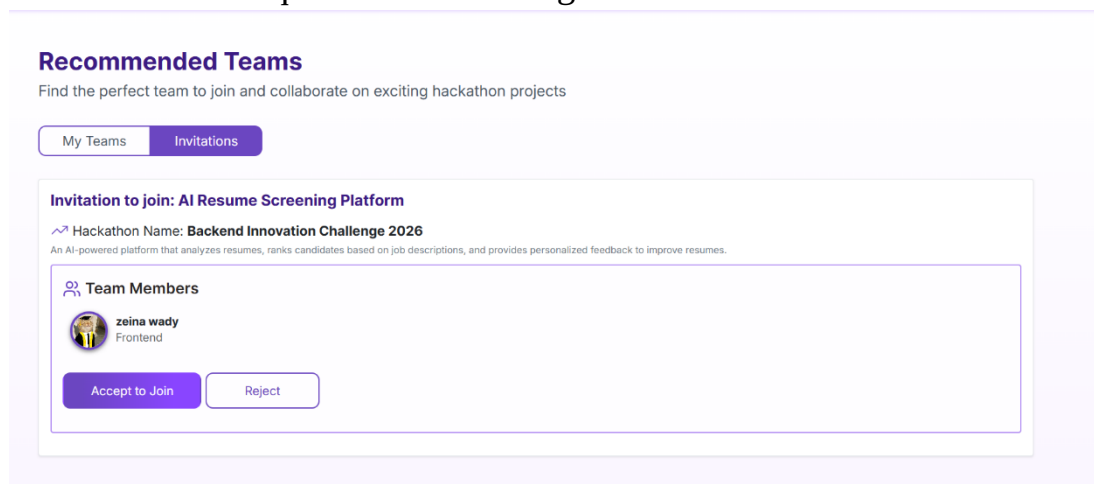


Figure 5.26: Team Invitations

5.14 Tracking Team Progress as an Invited Member

- Once a user accepts a team invitation, they gain access to the **Team Progress** page, where they can monitor the status of all other recommended team members.

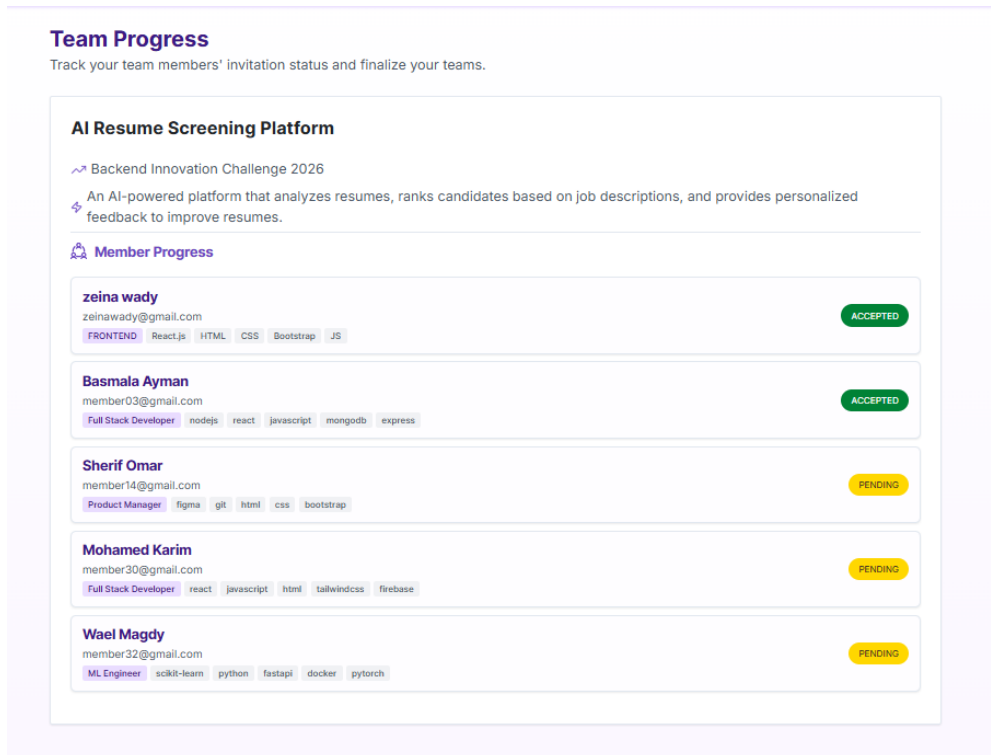


Figure 5.27: Tracking Team Progress as an Invited Member

5.15 Viewing Participant Profiles

- To help users make informed decisions, the platform allows them to view the profile of any participant before accepting a recommendation or responding to an invitation. By clicking on a participant's name, a pop-up window is displayed containing the participant's key information, including their profile picture, primary roles, skills, and professional links such as GitHub and LinkedIn.
- This feature is available both when reviewing AI-generated team recommendations and when responding to team invitations, allowing users to evaluate potential teammates and make confident decisions

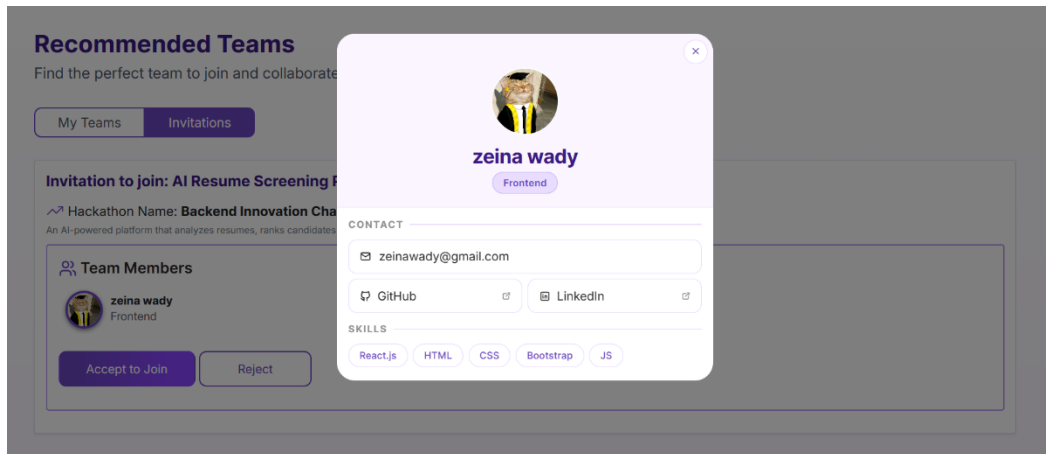


Figure 5.28: Viewing Participant Profiles

5.16 Team Rematching

- If the team owner is not satisfied with the current team, they can select **Find More Teammates** to trigger another round of AI-powered recommendations in order to complete the team. During this process, all teammates who have already **accepted** the invitation remain fixed in the team and are automatically included in the new recommendations. At the same time, members who **rejected** the previous invitation are excluded from future recommendations. The AI model then searches for new candidates to fill the remaining positions while preserving the accepted members, allowing the user to gradually build the desired team.

Team Progress

Track your team members' invitation status and finalize your teams.

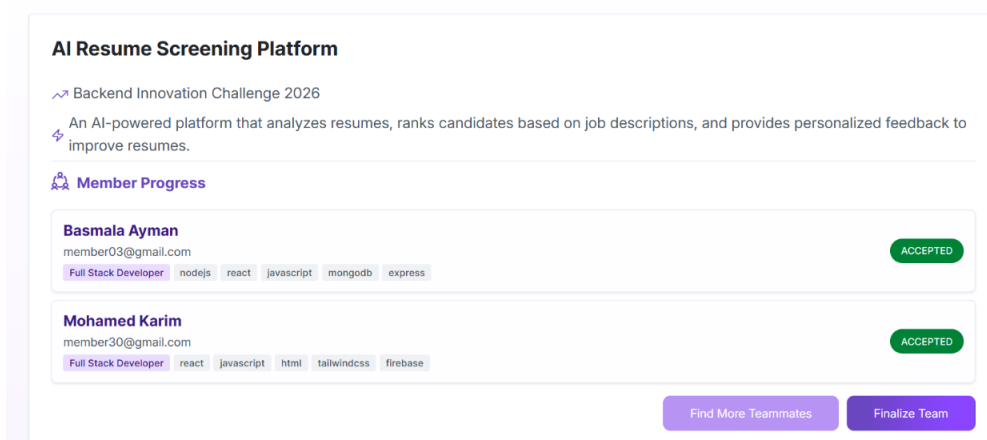


Figure 5.29: Team Rematching - 1

- Once the second round of AI matching is completed, the team owner receives a notification indicating that the new recommendations are ready.

Notifications

Stay updated with your teams and hackathons

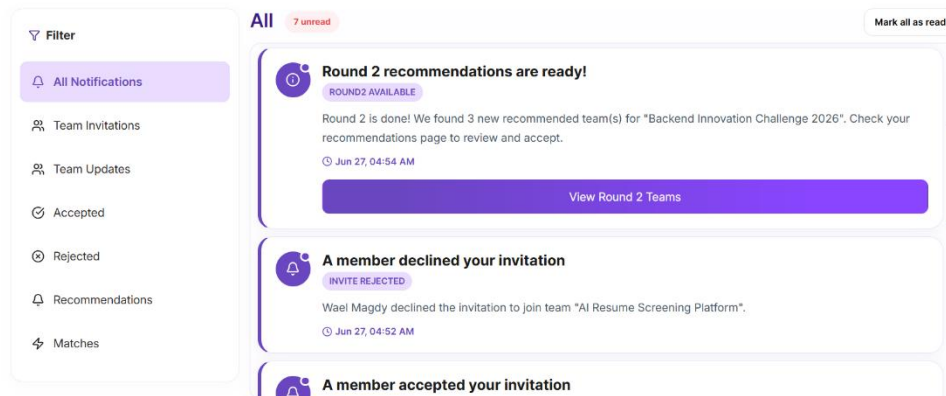


Figure 5.30: Team Rematching - Notification

- The newly generated team options are then displayed, allowing the user to review the updated combinations of teammates and select the team that best meets their requirements.

Recommended Teams

Find the perfect team to join and collaborate on exciting hackathon projects

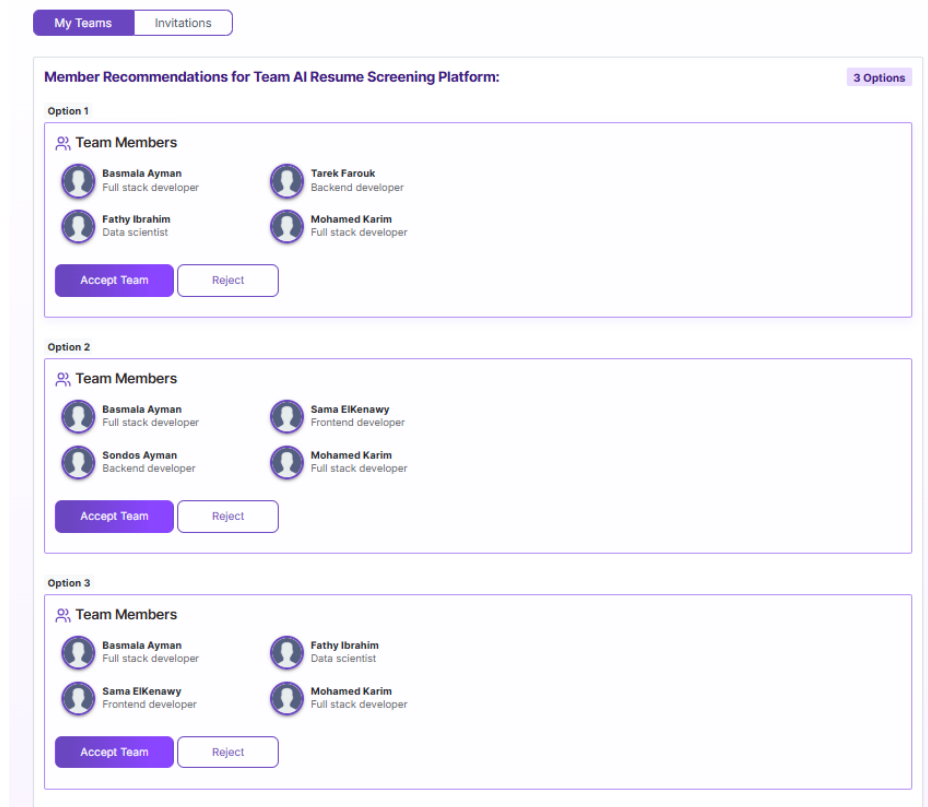


Figure 5.31: Team Rematching - 2

5.17 User Dashboard

- The Dashboard provides users with a centralized view of their activity on the platform, allowing them to quickly monitor their current progress and access important information.
- The dashboard displays key statistics, including the number of active teams, AI-recommended teams, and pending team invitations.

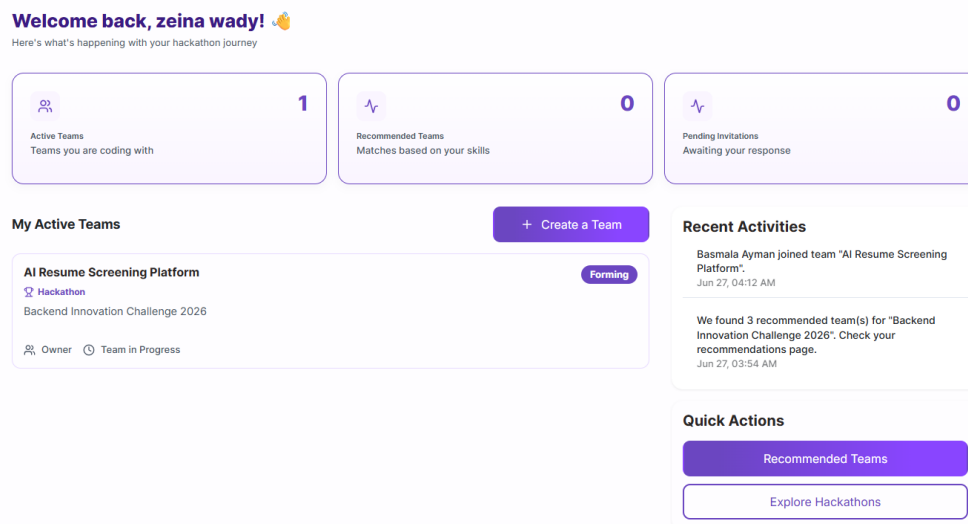


Figure 5.32: User Dashboard

5.18 Admin Dashboard

The Admin Dashboard provides administrators with a centralized view of the Team Catalyst platform, allowing them to monitor user engagement, team formation, and overall system performance through real-time analytics and interactive visualizations.

Key Features:

- Statistics Cards displaying the total number of registered users, created teams, and completed teams.
- Participant Growth line chart for tracking user growth over time.
- Team Formation donut chart showing the distribution of Auto-matched, Manual, and Invited teams.
- Weekly Activity bar chart comparing weekly user registrations and team creation.
- Recent Activities panel listing the latest platform events, including new user registrations, project creation, and other important updates.

The dashboard enables administrators to quickly monitor platform performance, identify trends, and make informed decisions based on real-time data.

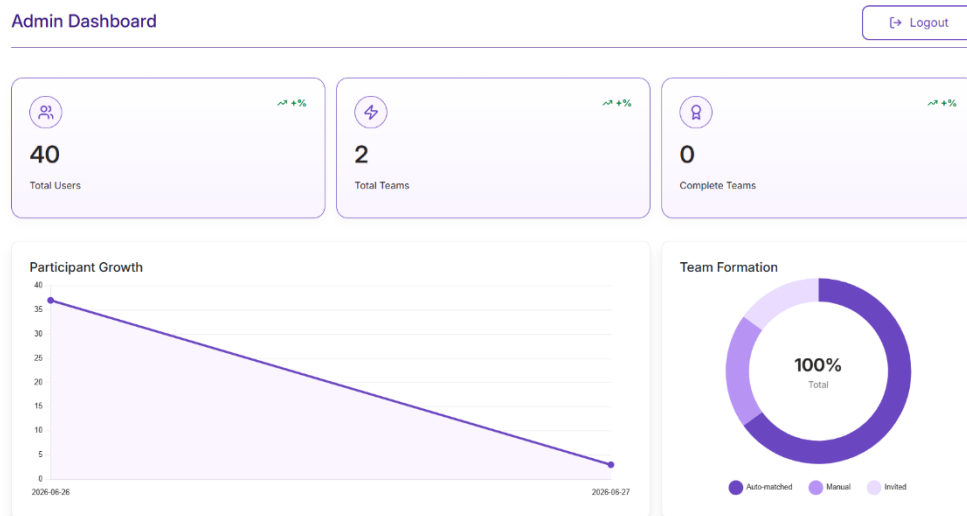


Figure 5.33: Admin Dashboard - 1

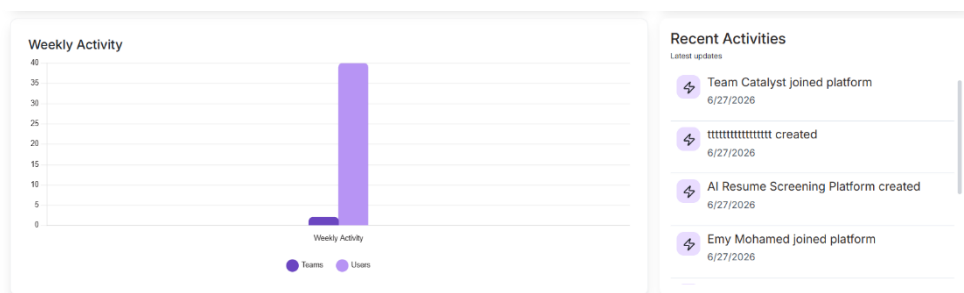


Figure 5.34: Admin Dashboard - 2

Chapter 6: Conclusions and Future Work

- 6.1 Conclusion.....72
- 6.2 Future Work72

Chapter 6: Conclusions and Future Work

6.1 Conclusion

Our project is a web-based intelligent hackathon team formation platform that helps users discover hackathons, create or join teams, and receive optimized team recommendations based on compatibility, relevance, and diversity. The system combines artificial intelligence and graph-based learning to improve the process of forming diverse and compatible hackathon teams.

We started by identifying the major challenges faced by participants during team formation, especially the difficulty of finding compatible teammates with complementary skills and shared interests. Based on these challenges, we designed the system architecture, collected and processed hackathon collaboration data, and developed an AI-powered recommendation pipeline adapted from the CsdRec framework.

The application offers several intelligent services, including:

- **Team Recommendation System:** generates optimized hackathon teams based on users' skills, interests, collaboration history, and project requirements.
- **Personalized Team Completion:** allows users to invite preferred teammates while enabling the system to intelligently complete the remaining team structure.
- **Project-Based Team Formation:** enables users to publish project ideas and receive compatible member recommendations from candidate applicants.

The final version of the application included features such as:

- A modern interface for exploring hackathons and project ideas.
- Greedy Beam Search-based team generation.
- Personalized team formation and team completion.
- Search and filter options for hackathons, projects, and teams.
- User profile and skills management for more accurate recommendations.

6.2 Future Work

There are several potential improvements and expansions that could significantly enhance the application in the future:

- **Granular Team Approval:** Empowering team owners with the flexibility to accept or reject individual members, rather than requiring bulk approval

for the entire team, in order to effectively streamline the team formation process.

- **Integrated Communication Hub:** Upgrading the platform to serve as a comprehensive workspace by facilitating centralized group chats, live virtual meetings, and private one-on-one messaging directly within the application.
- **Automated Skill Extraction:** Simplifying the onboarding experience by allowing users to upload their resumes, from which the system automatically extracts relevant technical and soft skills to save user time and effort.
- **End-to-End Hackathon Hosting:** Expanding the platform to manage the complete hackathon lifecycle, moving beyond event display to full native hosting that seamlessly handles team submissions, project tracking, and the final judging process.
- **Rating and Feedback System:** Establishing a performance evaluation framework for completed team collaborations based on peer-to-peer feedback, including a dedicated dashboard to display rating statistics and overall team evaluations.

References

- [1] Devpost , "Devpost - The home for hackathons," [Online]. Available: <https://devpost.com/>.
- [2] Y. G. Guo, "A reinforcement learning-assisted genetic programming algorithm for team formation problem considering person-job matching," *Information Sciences*, 2025.
- [3] B. Lv, "Team formation in large organizations: A deep learning approach considering skills and social networks," *Decision Support Systems*, 2024.
- [4] F. K. J. Y. H. Z. F. & T. C. Z. Fang, "Graph Enhanced Reinforcement Learning for Effective Group Formation in Collaborative Problem Solving," 2024.
- [5] A. D. Q. X. a. G. Y. J. Liu, "CsdRec: Accuracy and Diversity-Awared Team Recommendation for Collaborative Software Development," *IEEE Access*, vol. 11, pp. 67613-67623, 2023.
- [6] F. G. & D. M. P. Keane, "Using machine learning to predict links and improve Steiner tree solutions to team formation problems – a cross company study," *Applied Network Science*, vol. 5, 2020.
- [7] Hackathon Camp, [Online]. Available: <https://www.hackathon.camp/>.
- [8] Hackathon Party, [Online]. Available: <https://www.hackathonparty.com/>.
- [9] HackTribe, [Online]. Available: <https://hacktribe.co/>.
- [10] Devpost, "Devpost's hackathons API," [Online]. Available: <https://devpost.com/api/hackathons>.
- [11] A. A. F. a. A. W. N. Nurhidayat, "Predicting Outcomes of Hackathon Data Analysis," *Proceedings of the 2020 International Conference on Data Science and Its Applications (ICoDSA)*, 2020.